# University of Cincinnati

**Date: 7/28/2015**

**I, Chao Jin , hereby submit this original work as part of the requirements for the degree of Master of Science in Mechanical Engineering.**

It is entitled:

**A Sequential Process Monitoring Approach using Hidden Markov Model for Unobservable Process Drift**

Student's name: **Chao Jin**

This work and its defense approved by:

**Committee chair: Jay Lee, Ph.D.**

**Committee member: J. Kim, Ph.D.**

**Committee member: David Thompson, Ph.D.**

18409

# A Sequential Process Monitoring Approach using Hidden Markov Model for Unobservable Process Drift

A thesis submitted to the
Division of Research and Advanced Studies
of the University of Cincinnati

In partial fulfillment of the
requirements for the degree of
Master of Science

In the Department of Mechanical and Materials Engineering
of the College of Engineering and Applied Science

by

Chao Jin

2015

B.E. in Control Science & Engineering, Harbin Institute of Technology (2011)

Committee Chair: Dr. Jay Lee
Committee Member: Dr. Jay Kim
Committee Member: Dr. David Thompson

# Abstract

In the field of prognostics and health management, process monitoring is an essential technique to equip the system with the intelligence of being “aware” of any faults. Owing to tool fatigue, upstream material variation and electronic component drift, machine characteristics will often shift from initial states. As a result, sensor signals collected from the same equipment will possess varying correlation structures and offset in distributions, even if the health condition does not change. In order to build an effective data-driven process monitoring model, the constructed model has to be able to robustly differentiate the drifting healthy states from faulty conditions.

In this thesis, a sequential process monitoring approach using hidden Markov model is proposed for process monitoring to overcome influences of such drifts. During training stage, a discrete hidden Markov model is constructed using only healthy condition data. A health threshold is determined based on the deviation of normal condition health index, which is the normalized slope of negative log-likelihood. During monitoring stage, the health index of the new process from the same machine is calculated. Faults will be detected when the metric goes beyond the threshold. The developed approach has been validated using a case study for semiconductor etching process. And result of the proposed approach is benchmarked with both global and regime-specific local models using principal component analysis and self-organizing maps.

Copyright (c) 2015 by Chao Jin

## Acknowledgement

My sincerest and deepest appreciation goes to Prof. Jay Lee. In the beginning of 2012, after he accepted me as one of IMS, I knew that I have stepped onto the journey of relentless self-improvement. His philosophy of education and mentoring is never limited to success in academics, but always includes pursuit of professionalism and development of personality. He is a firm believer of the power of proactive attitude, and is himself practicing this belief. I could never imagine to possibly be half as strong-willed and perseverant as Prof. Lee, but he will always be the beacon of light in my life to remind me of higher goals.

I would like to thank the family of IMS. My special thanks goes to Dr. Edzel Lapira, Dr. David Siegel, Dr. Wenyu Zhao, and Mr. Shanhu Yang. They have been coaching me in study and life since my first day at IMS, and are role models in every aspect. I would like to thank Mr. Patrick Brown and Mr. Michael Lyons for patiently helping me in coordinating with industries, assisting with budgets and finances, writing professional emails, and teaching me how to work professionally. I would like to thank my same-year peers Mr. Eric Huang, Ms. Xiaorui Tong, Ms. Christina Lucas and Mr. Tyler Skirtich for creating a lot of nostalgic memories during our research studies. I would like to thank Mr. Zongchang Liu and Mr. Yuan Di for being supportive, extremely hard-working and inspirational. I would like to thank Mr. Zhe Shi for being the best roommate ever, and thank Mr. Xiaodong Jia and Mr. Matt Buzza for being great friends. I would like to thank Ms. Hung-An Kao for helping me in making presentations and

being such a great colleague to work with. I would like to thank Mr. Behrad Begheri for being inspirational in the perspective of creativity in graphic design and enthusiasm for technologies. I would like to thank all the other current and former IMS researchers: Dr. Yan Chen, Dr. Mohamed Abuali, Dr. Mohammad Rezvani, Dr. Wei Wang, Ms. Wenjing Jin, Mr. Hossein Ardakani, Mr. Arian Houshmand, Ms. Ellen Gamel, Ms. Laura Henkel, Mr. Dustin Brisset, Mr. Aaron Shelly, Mr. Brian Phillips, and Ms. Paige Weaver.

My greatest gratitude belongs to my family, especially my grandmother LEE Kyung-Suk, and my parents JIN Guangri and WEN Lianji. My grandmother has always put me on pedestal and tried to bring out the best of me ever since I was little. Their unconditional love and support have made me a man of good heart, and have been the guidance and eternal encouragement for me during my difficulties.

Last but not least, my beyond-word thankfulness goes to my girlfriend Ting-Yu Huang for her unwavering love and care. Locating at the opposite side of the earth, she has overcome the obstacles of loneliness and endless time differences, and has always had my back no matter what had happened. Her optimism has been the cure of my depression, and her inspiration is the medicine of my lack of confidence. It is our love that is making me a better person, and it is our love that motivates me to firmly move forward, for her, in the pursuit of happiness.

# Table of Contents

Abstract.....I

Acknowledgement.....III

List of Figures ..... VIII

List of Tables ..... XI

Chapter 1. Introduction .....1

1.1. Motivation .....2

1.2. Research Objectives .....5

1.3. Research Contributions.....7

1.4. Thesis Organization .....8

Chapter 2. Review of Related Literature .....10

2.1. Introduction.....10

2.2. Review of Common Data-Driven Process Monitoring Methods.....13

2.2.1. Principal Component Analysis .....15

2.2.2. Partial Least Squares.....22

2.2.3. Other Methods .....24

2.3. Review of Hidden Markov Model Application in Prognostics and Health Management .....27

2.3.1. Hidden Markov Model for Diagnostics and Prognostics.....29

2.3.2. Hidden Markov Model for Anomaly Detection and Process Monitoring ..... 32
2.4. Research Gaps ..... 35
Chapter 3. Hidden Markov Model-Based Robust Health Monitoring ..... 38
3.1. Introduction ..... 38
3.2. Problem Statement ..... 39
3.3. Methodology Overview ..... 42
3.4. Feature Extraction and Dimension Reduction ..... 44
3.4.1. Preprocessing ..... 44
3.4.2. Feature extraction ..... 45
3.4.3. Dimension reduction ..... 47
3.5. Vector Quantization ..... 48
3.6. Hidden Markov Model for Process Drift Modeling ..... 50
3.6.1. Theoretical Background ..... 51
3.6.2. Process Drift Modeling ..... 62
3.7. Health Metric ..... 63
Chapter 4. Case Study: Semiconductor Etching Process Monitoring ..... 65
4.1. Introduction ..... 65
4.2. Data Pre-processing, Feature Extraction, and Dimension Reduction ..... 66
4.3. Process Monitoring using Single Global Model ..... 70

4.4. Process Monitoring using Multi-Regime Local Model ..... 74
4.5. Application of Hidden Markov Model-Based Process Monitoring Approach ..... 79
4.5.1. Importance of Dimension Reduction ..... 79
4.5.2. Vector Quantization Evaluation ..... 80
4.5.3. Determining Model Structure ..... 82
4.5.4. Model Initialization and Training ..... 83
4.5.5. Monitoring Result ..... 85
4.6. Benchmarking and Discussion ..... 87
Chapter 5. Conclusions and Future Work ..... 90
5.1. Conclusions ..... 90
5.2. Recommendations for Future Work ..... 91
Bibliography ..... 93

# List of Figures

Figure 2-1 Distribution of computational learning methods in soft sensing (Kadlec et al., 2009) ..... 11

Figure 2-2 Illustration of principal components ..... 16

Figure 2-3 Scree plot example ..... 18

Figure 2-4 Partial least squares modeling scheme ..... 23

Figure 2-5 Illustration of artificial neural network ..... 25

Figure 2-6 Illustration of support vector machine ..... 27

Figure 3-1 Overall flowchart of the proposed robust process monitoring approach ..... 43

Figure 3-2 Illustration of vector quantization for continuous value discretization 50

Figure 3-3 Illustration of HMM process ..... 51

Figure 3-4 Three essential problems of HMM ..... 52

Figure 3-5 Illustration of $\alpha t(i)$ in the forward algorithm ..... 54

Figure 3-6 Illustration of forward algorithm at Time 2 ..... 55

Figure 3-7 Illustration of backward algorithm ..... 56

Figure 3-8 State sequence estimation from observation given the model ..... 57

Figure 3-9 Illustration of process drifting state modeling using HMM ..... 63

Figure 3-10 Illustration of log-likelihood slope for process monitoring (MOD, change to NLL) ..... 64

Figure 4-1 Differences in operation steps for trace signals ..... 69

Figure 4-2 Scree plot for LAM data PCA for number of PC selection ..... 70

Figure 4-3 Flowchart of single global model process monitoring approach ..... 71

Figure 4-4 Health monitoring for semiconductor etching process using PCA-MSPC ..... 72

Figure 4-5 Health monitoring for semiconductor etching process using SOM-MQE ..... 73

Figure 4-6 Flowchart of multi-regime local model approach for semiconductor etching process monitoring ..... 75

Figure 4-7 Health monitoring for semiconductor etching process using multi-regime local model (SOM + PCA) ..... 76

Figure 4-8 Health monitoring for semiconductor etching process using multi-regime local model (SOM + SOM) ..... 78

Figure 4-9 Vector quantization distortion rate evaluation to choose the appropriate number of symbols ..... 81

Figure 4-10 Bayesian information criterion evaluation for number of states selection ..... 83

Figure 4-11 Illustration of local optimum and global optimum ..... 84

Figure 4-12 Health monitoring for semiconductor etching process using HMM-based method ..... 86

Figure 4-13 ROC curve comparison between HMM-based health monitoring approach and other benchmarking approaches ..... 88

# List of Tables

Table 2-1 List of desired fault detection method characteristics ..... 12

Table 2-2 Summary of common data-driven process monitoring method ..... 13

Table 3-1 List of notations ..... 51

Table 3-2 Summary of forward algorithm ..... 56

Table 3-3 Summary of backward algorithm ..... 56

Table 4 Summary of Viterbi algorithm ..... 58

Table 5 Summary of Baum-Welch algorithm ..... 61

Table 4-1 Data description wafer run health conditions for LAM data ..... 65

Table 4-2 List of variables in LAM data ..... 67

Table 4-3 Summary of Data Pre-processing, Feature Extraction, and Dimension Reduction methods ..... 68

Table 4-4 Comparison of AUC ..... 88

# Chapter 1. Introduction

The manufacturing industry has been more automated since the introduction of information systems. Along with the advancement of production automation, sensing systems are generating a large number of process data from control systems or add-on sensors. It is believed that process data are closely related to asset health condition. To ensure product quality, control system will usually provide single-variable threshold to alarm faults based on physics. Meanwhile, process experts will inspect these parameters and infer asset health condition on the basis of domain knowledge and experience. Such common practices, however, cannot consistently offer accurate monitoring results, because faults sometimes are reflected in the changes of variable correlation rather than a single value of variables. Also, incipient faults might propagate and change the pattern of all the variables, but each single variable looks fine. And when single physical variable goes above threshold, the fault is already critical. Besides, human experience varies from one to another. Prognostics and health management (PHM) is a field dedicated to the research of asset health management by building advanced health monitoring models. Intelligent models will be able to detect abrupt system faults, assess asset health condition in terms of degradation, and predict asset remaining useful life (RUL) so that maintenance scheduling can be optimized and spare part inventory can be reasonably planned.

Among all the subfields of PHM, process health monitoring is an essential step to discover faults so that latter procedures such as diagnosis and prognosis are possible. There are two categories of health monitoring methods: one is physics-based modeling, and the other is data-driven modeling. Physics-based modeling for process industry requires mathematical description of ideal system steady states. Also, complicated mathematical modeling usually calls for significant amount of domain knowledge. The cost and time invested in building such a physical model might not always outweigh the return of revenue. These reasons render data-driven modeling gaining popularity for generalized fault detection methodology development and comparable, if not better, health monitoring accuracy. In this light, this thesis focuses on the development of a data-driven process monitoring approach with robustness against process drift.

## 1.1. Motivation

In the context of process monitoring, a fault is defined as an abnormal and undesirable deviation of system characteristics leading to unfulfilled operational purpose (Isermann, 1984). Fault detection is then the process to discover and report such deviation when it occurs. Data-driven fault detection methods build a baseline model from process history data, “memorize” the patterns, and detect faults by identifying faulty patterns or patterns that do no match with the healthy condition scenario. It does not need any prior knowledge of the monitored asset. However, as can be inferred from the name, one of the biggest challenges for data-driven methods to achieve desired effectiveness and accuracy is data quality. Process manufacturing data quality sometimes cannot meet the

requirements due to the following reasons. First, if the data are from automation system, then the sampling rate is usually low. Sampling frequency is especially vital for rotary machinery such as motors, gearboxes, and bearings. Low sampling frequency data pose a risk in data aliasing, rendering frequency domain analysis and feature extraction impossible. Also, there might be missing values in the sensor recordings. The value is usually empty, zero, or infinity. The causes of missing values can be complex, but the most common reason is sensor malfunctioning. Furthermore, outliers occur when the measurement deviates from interested or physically meaningful range. This is usually because sensor hardware failure or due to non-stationary changes in working conditions.

Another challenge that is related to data quality but not caused by sensor issues is posed by ambient disturbances. Ambient disturbances add additional uncertainty to sensor measurement, which results in an increase of false alarms. Sometimes, ambient disturbances have would change the characteristics and patterns of sensor signals, creating a new regime. For example, seasonal environment changes will affect wind speed and ambient temperature, thus influencing wind turbine fault detection if the model is not able to deal with seasonality. Also, ambient disturbances may cause similar faulty patterns so that false alarm would be generated. For instance, unbalanced supply in induction motors will generate similar patterns with winding faults. This will deceive the fault detection model and make it hard to distinguish real short-circuit faults from environmental changes.

Training data insufficiency is also a challenging issue for data-driven fault detection. Training data refer to the historical data samples used for data-driven model construction. In general, data-driven fault detection methods can be categorized into supervised and unsupervised types. For supervised methods, ideally a classification model should be trained using both healthy state and faulty state condition data. In this scenario, training data insufficiency usually means that there is a lack of completeness in faulty pattern spectrum in the training library, which makes the model unable to recognize certain uncovered fault types. For unsupervised methods, only healthy condition data are needed for model training. In this scenario, training data insufficiency is caused by occurrences of novel working regimes. For process manufacturing in particular, performance drift reflected in data is caused by variation of materials from upstream or drift in sensors. Regardless of model types, over-dependence on training data sufficiency and lack of robustness against regime changes will lead to significant decrease in fault detection rate and increase in false alarms.

In order to cope with the challenges mentioned above, improvements on data-driven fault detection methods have been proposed in academics. One of the solutions to remedying data quality issues is to perform more effective feature extraction. Methods such as symbolic dynamic feature extraction has been proposed (Sarkar, Jin and Ray, 2011) to model time series signals with predefined primitives and use the fitting parameters as the features, so that distortion caused by measurement noise and outliers will be reduced. The method is proved to be useful, but the computational efficiency of optimized

partitioning is challenging. Also, whether the captured symbolic parameters are useful in fault detection needs to be confirmed with process experts, which compromise one of the advantages of data-driven methods. Solutions to lack of robustness in fault detection models include methods such as using local models for the encountered regimes, introducing a correction factor in model parameters, or adaptively re-training of models when process drift is detected. These methods have improved the performance of conventional methods such as principal component analysis (PCA) or partial least square (PLS). However, existing methods that require local modeling and model competence is usually complex, which involves additional modeling of different regimes and might not be suitable for on-line implementation. Also, re-training of models for updates usually causes a delay for process monitoring. Besides, deviation in current modeling health metrics usually cannot be explained in physical senses, which gives less insight about the system.

In summary, data quality and model robustness against process drift remain to be two of the major research gaps in data-driven fault detection methods. This motivates the research study proposed in this thesis to develop a sequential robust fault detection methodology against process drift with insufficient training data using hidden Markov model.

## 1.2. Research Objectives

The objective of this research study is to develop a sequential robust fault detection approach. The targeted major research gap is the lack of model robustness due to process drifts and insufficient training data.

The proposed methodology in general is to build a unified model using normal condition data and to calculate the hidden Markov model log-likelihood of the incoming data under the constructed model. If the data (observations) are from asset healthy condition, then the log-likelihood will steadily decrease, as the sequence length gets longer. Yet if the data are from asset faulty condition, then the log-likelihood will decrease faster than the normal case scenario. Overall, the proposed methodology is realized by three major steps. The first step is to transform the feature space into sequences of discrete vectors. The second step is to build a hidden Markov model using normal condition sequences. The third step is to evaluate incoming sequence log-likelihood and compare the decreasing speed with the normal condition case.

The proposed methodology is to be realized by completing the research tasks listed as follows:

a) Formulate an optimized procedure to preprocess features before HMM training – reduce feature space dimension, ensure feature independence, discretize feature vectors and evaluate the effectiveness of discretization.
b) Develop a method to optimize the construction of hidden Markov model – issues of initialization, model structure selection, etc.
c) Develop a health metric that is able to accurately reflect asset health condition change – log-likelihood slope sequence inclusion and quantization of slope decreasing speed change.

d) Evaluate the developed methods and benchmark with existing widely used methods – application of the methodology on an industrial data set and performance comparison with other approaches.

## 1.3. Research Contributions

The contributions of this research study can be categorized into academics and industrial practices:

For academics, this research study proposes a novel thinking in process fault detection modeling:

1) In this research, the actual modeling subject is a sequence of discretized values transformed from continuous feature space. The model is therefore recognizing the sequence of cluster labels rather than the actual values. This adds robustness in modeling: even if there is a small surge in data distribution or abrupt abnormal values, as long as the discretized sequence is consistent with the normal condition, the model will not be ill-influenced.

2) A unified process monitoring approach using hidden Markov model is formulated and validated. Even though hidden Markov model is also seen to be used for process monitoring, details of how to select model structure, how to prepare the data for the model, and how to evaluate the effectiveness of model are rarely seen. This research provides a systematic procedure in constructing process monitoring hidden Markov model, with care on fulfilling model assumptions.

For industrial application, the proposed methodology benefits the industry in the following aspects:

1) The developed approach makes it possible to achieve significantly better process monitoring performance using limited normal condition data for semiconductor fabrication monitoring.

2) The proposed approach can also be applied on other industries such as oil and gas, chemical processes, etc. The approach can be generalized for modeling of other sequential time-series, or for event sequence mining problemes.

## 1.4. Thesis Organization

A review of related literature is provided in Chapter 2. Recent development in data-driven process monitoring approaches have been investigated and reviewed. The use of hidden Markov model in prognostics and health management is also summarized.

Chapter 3 presents and elaborates on the formulation of the proposed robust sequential process monitoring approach. An overview of the method is introduced first, followed by a detailed explanation of the principles and characteristics of hidden Markov models. Then the method of feature space discretization is formulated. Finally, the health metric derived from log-likelihood is presented, with explanation on the meaning in physical sense.

Chapter 4 evaluates the proposed methodology on a semiconductor etching process. Other techniques using principal component analysis – statistical

process control and support vector machine are also introduced. The performances of the three approaches are then benchmarked and discussed.

Conclusion of this research study is drawn in Chapter 5. Advantages and limitations are discussed in this section, and future works are also recommended to deepen the research on robust process monitoring and other usages of hidden Markov model in prognostics and health management.

# Chapter 2. Review of Related Literature

## 2.1. Introduction

Fault detection for process monitoring by definition is the process of recognizing unacceptable deviation of a characteristic property that causes failure of accomplishing operations. In general, fault detection methods can be categorized into two families: one is model-based, and the other is data-driven. Data-driven fault detection, compared to model-based methods, is gaining popularity in recent years due to the increasing complexity of the system which requires deep domain knowledge to construct a physical model, and the excellent performance when applied appropriately. For process monitoring in particular, fault detection is sometimes also referred to as the technique of “Soft Sensor”, indicating the health index generated by fault detection algorithm to determine equipment health. An excellent survey has been conducted by (Kadlec, Gabrys and Strandt, 2009), bringing about a summary of popular algorithms being used for process fault detection, which is shown in the following referenced pie chart.

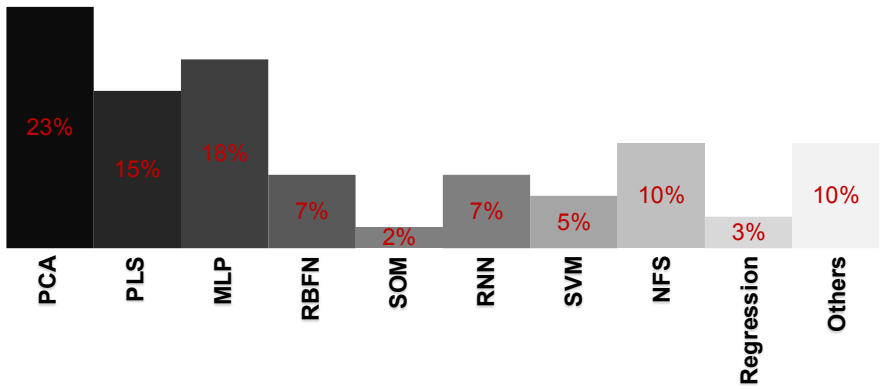



*Figure 2-1 Distribution of computational learning methods in soft sensing (Kadlec et al., 2009)*

It can be seen from the pie chart that multivariate statistical analyses like principal component analysis (PCA) and partial least square (PLS) together compose about 40% of all the technique applied for process monitoring, artificial neural network (ANN) based methods, such as multi-layer perceptron (MLP) and self-organizing maps (SOM), contributes to another around 35%, and other techniques such as support vector machine (SVM) belong to the rest. Thanks to the multidisciplinary nature of data-driven fault detection, choices of algorithms are greatly impacted by the development of machine learning techniques. Application of newly proposed techniques or existing techniques that have received less attention in processing monitoring are usually where improvements take place.

Although the mechanism of each algorithm varies, they share some common methodology when being applied for fault detection, as depicted in the following flowchart. Typically, signals (or raw data, which will be referred to interchangeably in this thesis) collected from sensors will first go through a series of pre-processing procedures to ensure that the processed data are in an

appropriate format for the algorithm of choice. Note that the procedure of feature extraction is included in this step for generality, since feature extraction can be described as a transformation to the appropriate information-rich space from the original raw data space. Then, the algorithm will take in the processed data and fuse the multidimensional information into a single-dimensional metric. For unsupervised methods, during training stage, a health threshold is obtained from the normal condition data. During monitoring stage, the metric of the monitored equipment will be calculated and compared to the threshold. Whenever the metric exceeds the threshold, the equipment condition is considered to be faulty. For supervised methods, during training stage, healthy condition and faulty condition samples will be categorized into two classes. During monitoring stage, the monitored equipment data will be classified into the predefined categories for the system to be able to recognize the health condition.

Given the diversity of the techniques and metrics, an evaluation system is needed in order to benchmark different methods and make improvements. A series of desirable characteristics for fault diagnostic algorithms have been proposed by (Venkatasubramanian, Rengaswamy, Yin and Kavuri, 2003). Since fault detection and fault diagnosis systems share some common properties, these desired characteristics are selected and slightly modified for fault detection and briefly listed in the following table.

*Table 2-1 List of desired fault detection method characteristics*

| # | **Characteristic** | **Remark** |
|---|---|---|
| 1 | Quick detection | Fault detection system should be sensitive to failures on condition that false alarm number is |

| | | |
|---|---|---|
| | | minimized. |
| 2 | Robustness | Fault detection system should be robust to noise and uncertainties, and the performance should degrade gradually instead of totally fail. |
| 3 | Novelty identifiability | Fault detection system should have the ability to distinguish novel malfunction from known faults and normal conditions. |
| 4 | Detection error estimate | A prior estimate for fault detection rate will facilitate the building of application confidence. |
| 5 | Adaptability | The ability of gradually developing the scope of the system as new working conditions occur. |
| 6 | Explanation facility | The ability of explaining how the fault originated and propagated to malfunction. |
| 7 | Modeling requirements | Modeling effort should be minimized. |
| 8 | Storage and computational requirements | Real-time monitoring system requires that the algorithm to be computationally efficient. |
| 9 | Multiple fault identifiability | The ability to detect multiple faults occurring at the same time. |

By applying the abovementioned nine metrics, it is possible to benchmark and evaluate the “goodness of fault detection systems”, which will be performed in Section 4.6. of this research study.

In the following sections, a series of algorithms widely adopted in process monitoring fault detection are described with their pros and cons discussed. This is followed by a review on hidden Markov model application in prognostics and health monitoring, fault detection in particular. Eventually, research gaps are pointed out and the rationale for proposing this research study in the first place is introduced.

## 2.2. Review of Common Data-Driven Process Monitoring Methods

*Table 2-2 Summary of common data-driven process monitoring method*

| **Family** | **Algorithm** | **Pros and Cons** |
|---|---|---|
| Principal | PCA-MSPC | Pros: easy to implement. It only needs |

| | | |
|---|---|---|
| Component Analysis | | nominal condition data for training.<br>Cons: the model is static, not able to model process drift; can only model linear relationship between variables. |
| | RPCA | Pros: able to update model parameters when condition changes.<br>Cons: the computational complexity is a challenge as sample number gets larger and larger even though efficient algorithms such as “Lanczos tridiagonalization” was applied; not able to handle nonlinear relations. |
| | MWPCA | Pros: improves the computational problem of RPCA by updating the parameters using only a window of data.<br>Cons: the choice of window size is crucial to the performance of the algorithm; not able to handle nonlinear relations. |
| | DPCA | Pros: able to update PCA parameters by modeling the linear relation between adjacent data samples.<br>Cons: requires the selection of window length; not able to handle nonlinear relations. |
| | KPCA | Pros: kernel trick is applied to model nonlinear relations between variables.<br>Cons: static model that cannot be adaptive to process changes. |
| Partial Least Square | PLS | Pros: unlike PCA, PLS is measuring the change of input variables according to the changes of output variables, which is suitable for systems that have quality variables to estimate.<br>Cons: static model; not able to model nonlinear relations; unlike PCA, the model is harder to interpret. |
| | RPLS, MWPLS | Pros: recursively updating PLS parameters to equip PLS with adaptivivity.<br>Cons: RPLS has the issue of computational complexity; MWPLS is sensitive to window |

| | | |
|---|---|---|
| | | length selection; not able to model nonlinearity. |
| | Multi-way PLS | Pros: able to model nonlinear relationships between variables.<br>Cons: static model that lacks adaptivity. |
| Neural Network | ANN | Pros: able to model nonlinear relations between variables.<br>Cons: easy to converge at local minimum and leads to unstable result for each run; not able to adapt to process drifts. |
| | SOM-MQE | Pros: able to model nonlinear relations and the result is much more stable.<br>Cons: not able to adapt to process drifts. |
| Others | SVM | Pros: able to model nonlinear relations; computationally efficient.<br>Cons: not able to adapt to process drifts. |

### 2.2.1.Principal Component Analysis

Principal component analysis (PCA) is one of the most commonly used algorithms for process monitoring (Ku, Storer and Georgakis, 1995; Schölkopf, Smola and Müller, 1997; Wise, Gallagher, Butler, White and Barna, 1999; Li, Yue, Valle-Cervantes and Qin, 2000; Jolliffe, 2002; Smith, 2002) attributing to its simplicity, unsupervised nature, and decent performance. It is a tool itself and the basis for other more complicated algorithms such as auto-associative neural network. Basically, PCA is a dimension reduction technique that is able to retain most of the variation (information) in multi-dimensional data (Jolliffe, 2002). The so-called principal components are the underlying “directions” where the data are

spread out the most, as shown in Figure 2-2. Such directions are picked up by PCA and used as the “structure” that can represent the data.

The procedures of applying PCA are as follows (Smith, 2002):

1) Normalization so that each variable has zero mean and unit variance;
2) Obtain the covariance matrix;
3) Calculate the eigenvectors and eigenvalues of the covariance matrix;
4) Determine the number of principal components (PC) to keep; and
5) Project the original data into PC space.

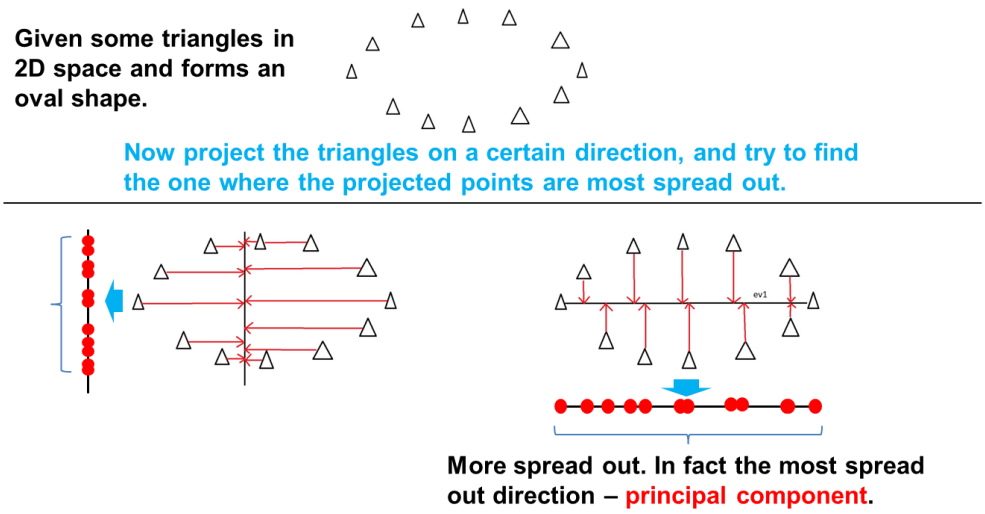



*Figure 2-2 Illustration of principal components*

When applying PCA for fault detection, statistics such as Hotelling’s $T^2$ and squared prediction error (SPE) are adopted as the health index to monitor whether the data set is deviating from the normal condition. The health limit of T2 and SPE statistics are based on the distribution of each of them, and usually a 99% or 95% confidence limit is calculated as the threshold. The application of

this type of method is referred to as PCA-MSPC (multivariate statistical process control).

One critical issue of applying PCA is the selection of the number of PCs. There are three known methods:

1) Scree plot
2) Kaiser-One
3) Cumulative variation proportion

Scree plot is a method that plots all the sorted eigenvalues to visually inspect the number of eigenvalues that contain majority of the information. It is assumed that there will be a “break” between the eigenvalues that contain large variation and the ones that do not, as shown in Figure 2-3. In this example, before the break, the eigenvalues contain most of the variation; after the break, it will not make too much difference if more eigenvalues are included. Therefore, only two eigenvalues are chosen in this case. However, sometimes there is no break in the eigenvalues and each PC shares a fairly uniform amount of variation. In such scenarios, Kaiser One and variation percentile methods can be used. The rationale behind Kaiser One method is that, since the data has already been normalized to unit variance, any PC that has variance bigger than one should be regarded as significant. So a threshold of one is applied to select any of those PCs possessing an eigenvalue greater than one. For variation percentile method, eigenvalues need to be sorted in descending order first, then select PCs that contain $\alpha\%$ of the total variation, where $\alpha$ is defined by the users.

Basic PCA-MSPC technique already offers the benefits of unsupervised training and decent performance. Also, another advantage of PCA is that it can generate the contribution plot for each of the input variables to diagnosis which signal has a more significant impact on the variation. However, the drawbacks of PCA-MSPC technique are that it cannot adapt to the changes or drifts of the system and can only model linear relationships, which will trigger an increasing number of false alarms over time. To improve the performance of PCA, variations have been created, which will be introduced in the rest of this subsection.

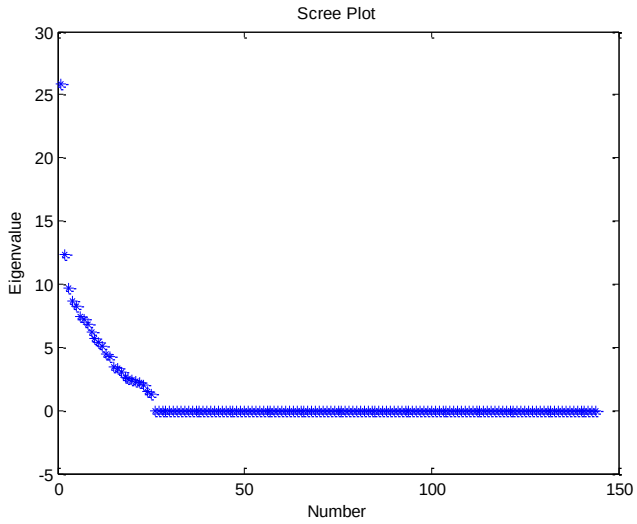



*Figure 2-3 Scree plot example*

### *Recursive PCA*

Recursive PCA is a technique developed to make PCA more adaptive to the changing working conditions in order to increase the robustness of the model. The general idea is to update the PCA model, which includes: (1) the covariance matrix, (2) the healthy threshold, and (3) the principal components and the

number of them. The overall scheme of updating the model can be summarized as follows (Li et al., 2000):

1) Recursively update the mean for the covariance matrix after including a new sample;
2) Efficiently calculate PCA models;
3) Recursively determine the number of principal components, and the significant number of components changes over each iteration; and
4) Recursively determine the healthy thresholds based on $T^2$ or SPE statistics in real-time to facilitate adaptive monitoring.

Li et al (2000) proposed two recursive PCA algorithms to incrementally update the PCA model based on the original model structure, and validated the technique using a thermal annealing process. The method was also proved to be robust against missing values and outliers.

***Moving-Window PCA***

Another somewhat similar method is referred to as moving-window PCA. Recursive PCA updates the correlation matrix by adding a new sample to its current value, and yet moving-window PCA updates the correlation matrix by getting rid of the first sample from the correlation matrix, and then adding a new sample to the matrix. However, the conventional moving-window PCA consists of two steps for modeling: elimination of old sample and inclusion of new sample. An improved moving-window PCA method takes advantage of both of the aforementioned methods, and derives a set of equations to directly calculate from the original matrix to the updated matrix which eliminates the oldest sample

and includes the new sample. This method is referred to as fast Moving-window PCA (MWPCA), proposed by Wang (2005). Then, for process monitoring, an N-Step-Ahead prediction method is utilized, which is another improvement made over conventional MWPCA. For any moving-window method, the choice of the window size will have a great influence on the performance. Wang (Wang et al.) proposed to use a sensitivity analysis for selection of window-size.

***Dynamic PCA***

Another effort to extend the effective but static PCA method for dynamic time-series monitoring is the dynamic PCA (DPCA) proposed by Ku et al (1995). Multivariate process time-series are always redundant, correlated and influenced by noises. PCA is able to extract the components that bear the maximum amount of information (variation) from the monitored system, but noise will affect the result and lead to false positives. To deal with noises, it is suggested that time-series modeling methods like autoregressive integrated moving average can be used (George, 1994). It models the next measurement point based on previous history, and generate a one-step-ahead estimation residual to monitor the variable. Nevertheless, such univariate prediction method presumes that the variables are not cross-correlated, which might not always be true in practice.

Basically, the idea of modeling the dynamically changing correlation and shifted mean of system time-series is to model the linear relations between adjacent time-varying terms, $X(t)$ and $X(t-1)$. If we assume that the dynamic relationships are first order system, then the relation will be the noise subspace of the following equation (Ku et al., 1995):

$$[X(k)X(k-1)]b = 0$$

where $X$ represents the data matrix, $k$ is the current timestamp, and $b$ is the singular vector. More generally speaking, the equation can be written as (Ku et al., 1995)

$$X_A(l)b = 0$$

where

$$X_A(l) = [X(k)X(k-1) \dots X(k-l)]$$

By appropriately selecting the window length $l$, both the static and the dynamic relations are expected to occur in the noise subspace with small singular values. This methods assumes that there is already a mature way of choosing the optimal number of principal components at each time of the calculation.

***Kernel PCA***

So far, all the mentioned PCA methods model the linear correlation relationship between different variables or features. In reality, it is not rare that the relationship is nonlinear. For such scenarios, techniques like nonlinear PCA has been proposed. Among all the nonlinear PCA methods, kernel PCA is one of the most representative and popular technique, given the popularity in applying kernel function in other methods like regression. It is believed that kernel PCA is first proposed in 1997 by Scholkopf et al (1997). The setting of the orthogonal projection of original data to get the principal components is generalized into a nonlinear one. The data will be firstly projected nonlinearly into a feature space with arbitrarily number of dimensions, and then PCA is performed on the new

feature space. It is realized by using the kernel technique similarly use in Support Vector Machine (Boser, Guyon and Vapnik, 1992). Different types of kernel functions (sigmoid, polynomial, etc.) can be used to model the nonlinear relationship between variables.

### 2.2.2.Partial Least Squares

Partial least squares (PLS) models are another type of methods that have been widely implemented for process monitoring. It is used when the data dimension is high and correlation is present. PLS is officially proposed by Herman Wold (Wold, 1985) for the analysis of economics. Once it was introduced to the community of process monitoring, it suddenly became popular. Like principal component analysis, PLS also projects the data into another space and extract the underlying vectors that represent certain properties of the original data and amplify them. Yet unlike principal component analysis, instead of measuring the data space variance, PLS finds how much variance input data causes according to the changes of output data. As shown in Figure 2-4, the mathematical expression of the model is as follows:

$$X = TP' + E$$

$$Y = TQ' + F$$

where $T = [t_1, t_2, \dots, t_n]$ is the score matrix, $P = [p_1, p_2, \dots, p_n]$ is the loading matrix for input data $\mathrm{X}$, and $\mathrm{Q} = [\mathrm{q}_1, \mathrm{q}_2, \dots, \mathrm{q}_\mathrm{n}]$ is the loading matrix for output $\mathrm{Y}$. $\mathrm{E}$ and $\mathrm{F}$ are residual matrices, which are to be minimized during modeling. In order to estimate the model parameters and take in new data samples, the direct weight matrix is needed. It can be obtained through the following equations (Wold, 1985):

$$T = XR$$

$$R = W(P'W)^{-1}$$

where $W$ is the weight matrix. The projected data from both input and output space are as follows (Qin, 2012):

$$\hat{X} = PR'X$$

$$Y = QR'X$$

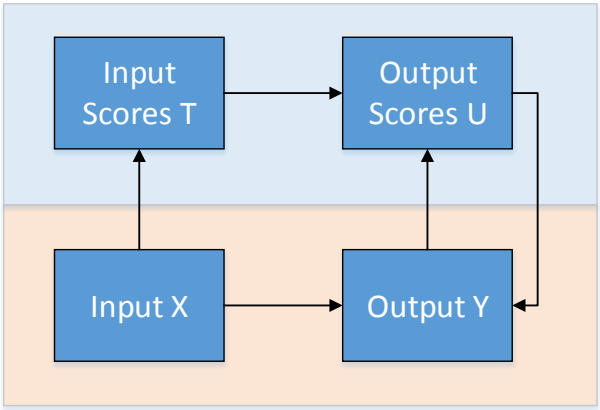



*Figure 2-4 Partial least squares modeling scheme*

***Extensions of PLS***

Similar to PCA, PLS only models the linear relationship between the input data according to the output data, and the original form of the model is designed for static state rather than non-stationary processes. As a result, the efforts of improving PLS are similar to PCA based methods as well: recursive PLS (Qin, 1998) and moving-window exponentially weighted PLS (Dayal & MacGregor, 1997) are proposed to equip the model with adaptivity; nonlinear PLS including methods like neural network PLS (Qin & McAvoy, 1992) and multi-way PLS (Bro, 1996) are designed to tackle the issue of nonlinearity. The exact procedure of

how to carry out such methods are omitted here due to the similarity between the implementation of PCA and PLS. In general, PLS based methods are also able to compute the contribution of variation source that causes the change of output, and therefore perform basic root cause of the problem or effective sensor selection.

### 2.2.3.Other Methods

#### *Feedforward Neural Networks*

Feedforward neural networks are also known as multi-layer perception (MLP). The so-called neural net is composed of a series of logistic regression models stacked on one another in the middle several hidden layers, and with the final layer to be another logistic regression model or linear regression model depending on whether classification or regression are needed (Murphy, 2012). It is a very useful algorithm widely used in handwriting recognition, rule-based programming, speech recognition, computer vision, etc. It is widely referred to as “neural” because the original intention was to imitate the information processing behavior of the human neural system.

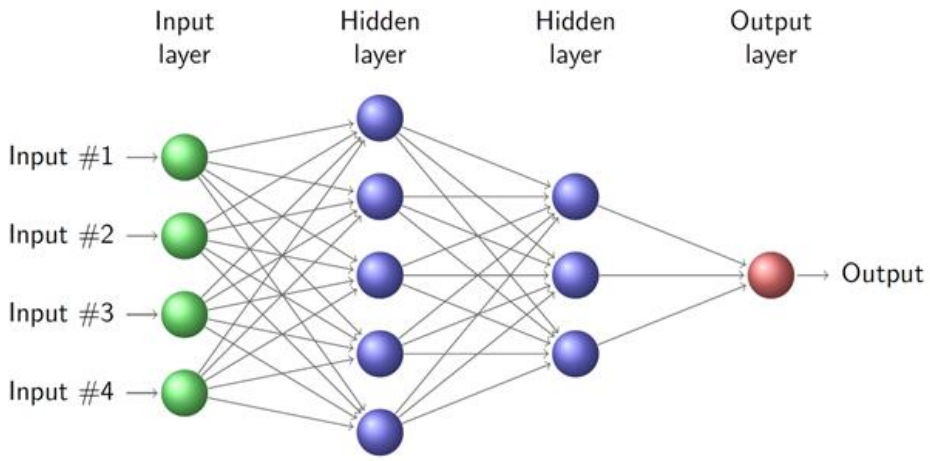



*Figure 2-5 Illustration of artificial neural network*

*(Image courtesy: http://mewarnai.us/160149-artificial-neural-network)*

The power of neural networks lies in its generality. The flexible and rich structure of the network makes it a “universal estimator” to model any smooth functions with reasonable accuracy, given an appropriate number of hidden units (Hornik, 1991). There are variants being widely used specifically useful for different contexts: recursive neural networks (RNNs) have a feed-back mechanism and are able to model temporal time-series information which can be a common repetitive pattern in industrial process signals; self-organizing map (SOM) that is able to deal with unsupervised problems and offers excellent visualization; auto-associative neural networks (AANN) model the nonlinear relationship between different variables and are outstandingly useful in modeling time-series multidimensional data.

The drawbacks of the feedforward neural network method are almost as obvious as its advantages. One of them is the training of a neural network usually falls into local minima, leading to inconsistent result for each run of the program. Another disadvantage is that it is difficult to find an optimal structure of the neural

net. The choice of the number of hidden layers sometimes can only depend on certain measures of “goodness-of-fit”. Besides, due to the unknown structure of the trained network, it is hard to explain any physical meaning out of the model or the result (Kadlec et al., 2009).

***Support Vector Machine***

Support vector machine (SVM) has been very popular in a wide spectrum of applications since its occurrence in the 1990s (Suykens & Vandewalle, 1999). Although it is a nonlinear method originally designed for binary classification, and yet it can also be modified for regression and multi-classification as well. In the subsection that discusses about kernel PCA, it is already mentioned that SVM is a kernel-based method. Basically, an $l_2$ regularized empirical risk function is to be minimized by replacing the terms to a form that only involves inner products – which we refer to as the “kernel” function. Also, the solution has to be made sparse so that the prediction only depends on a subset of the training data (support vectors). This is realized by changing the loss functions other than quadratic or log-loss (Murphy, 2012).

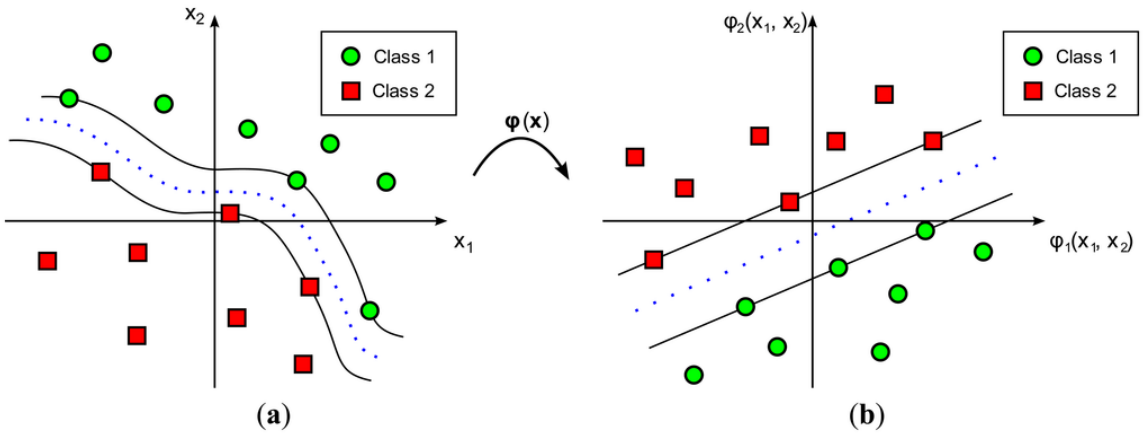



*Figure 2-6 Illustration of support vector machine*

*(Image courtesy: http://i.imgur.com/WuxyO.png)*

SVM is computationally efficiently compared to probabilistic methods (Kadlec et al., 2009). However, when the dimension of data gets too large, this advantage may not hold anymore. Also, although SVMs can be modified for regression or multi-class kernel-based classification, there are other techniques like $l_1$ - regluarized vector machine (LIVM) or relevance vector machine (RVM) (Tipping, 2001) that works even better. Particularly to process monitoring, the application of SVM is still to be explored to maximize its advantages.

## 2.3. Review of Hidden Markov Model Application in Prognostics and Health Management

| Application | Model Type | Highlight |
|---|---|---|
| Diagnostics and Prognostics | Discrete HMM | Unified fault detection & severity identification through log-likelihood evaluation. |
| | Continuous HMM | Unified diagnosis (evaluation of log-likelihood) & prognosis (estimate duration until state transition). |

| | | |
|---|---|---|
| | Continuous HMM | Autonomous diagnosis by evaluating log-likelihood for sequential clustering. |
| | HSMM | Modified structure is more accurate in prediction. Both diagnostics and prognostics are included. |
| | MoG-HMM | More accurate in prediction than HMM and HSMM. |
| Anomaly Detection | Continuous HMM | Incorporate mean-time-to-failure into the transition matrix probability to better model the faults. |
| | Continuous HMM | Visualization of faulty condition HMM log-likelihood deviation from normal condition. |
| Process Monitoring | PCA/ICA + HMM | Use NLL + MD as the health metric to consider both global and local probability changes. |
| | HMM + ICA | Use HMM as the operating condition identifier, and build local ICA model for process monitoring. |
| | Regime-specific complex HMM | Degradation is represented by HMM states; the starting state of this HMM is the ending state of the previous HMM; model search through GA; health metric represented by normalized log-likelihood slope. |
| | HMM + SPAMM | Both HMM and SPAMM model is built for monitoring, but the metric is switched based on the better model. |

Hidden Markov model (HMM), as the name suggests, is a probabilistic algorithm that contains two stochastic processes linked by a probability distribution. It became a widely used tool for all kinds of applications ranging from biomedical to speech recognition due to its rich and extendible mathematical

structure and the robustness and effectiveness when applied properly. PHM applications are also starting to adopt the method in the past two decades to benchmark with existing algorithms and making improvements. The following subsections will try to take an overview of HMM applications in the field of PHM and summarize the current common practice and tricks.

### 2.3.1.Hidden Markov Model for Diagnostics and Prognostics

A hidden Markov model has the structure of an observation model governed by an underlying Markov process. This unique structure makes it possible to include diagnostics (fault classification) and prognostics (prediction) in one single model. Usually, one HMM is trained using data from a particular health state, normal or faulty. Then each faulty state model will be having the left-to-right state structure to model the degradation process. And after recognizing the current health state category by evaluating the likelihood of the monitored data with regard to each trained HMM, the remaining useful life can be predicted by estimating the duration left until reaching the final state – failure of the model.

Specifically, Kwan et al (2003) proposed a method for unified fault diagnostics and fault severity recognition, which can serve as the foundation for prognostics. Discrete HMM is used. The collected continuous vibration data from a DC motor test bed are preprocessed using frequency domain techniques, and then PCA is performed to reduce the dimension of feature space. Afterwards, several HMMs are trained for each fault type. During monitoring stage, fault type is identified by comparing the scores (likelihoods) of each HMM, and then the fault propagation profile is obtained by evaluation of HMM state sequence.

Baruah and Chinnam (2005) also proposed a unified method for diagnostics and prognostics in machining process. Similar to Kwan's work (Kwan et al., 2003), the diagnostic classifier is also trained using one HMM for one health condition. For prognostics, the authors took advantage of the HMM state duration by estimating the state transition time distribution. So the mechanism for prediction would take two steps: 1) estimate the current health state by evaluating the maximum likelihood of the observed sequence; 2) estimate the time till next transition or till the transition to failure by calculating the conditional probability distribution of a distinct state transition given the previous state transition points. The proposed method was validated using a machining drilling process. At the end of the article, the authors also discussed about the proper handling of HMM for PHM application.

Kumar et al (2008) took a step forward and proposed an autonomous diagnostics approach using HMM sequential clustering. Instead of training an HMM using each health condition, an unsupervised training which only requires healthy condition data is designed. A threshold is predefined on health condition loglikelihood, and if the new observation loglikelihood falls below that threshold, it will be considered as a new health condition, and a new HMM will be trained. This allows the model to automatically detect the degradation stage of the system, and the method was validated on a drilling process.

Dong et al (2006) proposed a new method called "hidden semi-Markov models" (HSMM). The novel method modified the Markov state transition structure and let a state generate a sequence of observation instead of just one.

This modification has enabled the model with flexible state duration instead of exponential, which makes prognostics more accurate. Unified framework of both diagnostics and prognostics are proposed and tested on a transmission planetary carrier.

Tobon-Mejia et al (2011) compared different types of HMM for bearing degradation prognostics. Mixture of Gaussian HMM (MoG-HMM) performance was proved to be better than HMM and HSMM for bearing remaining useful life prediction. And in a latter research article (Tobon-Mejia, Medjaher, Zerhouni and Tripot, 2012), details of how MoG-HMM is implemented is discussed. Basically, several MoG-HMMs are trained using different observation sequences with different duration and initial states. During monitoring stage, the newly observed sequence is used to calculate the conditional probability for each trained MoG-HMM, and the one that has the highest probability will be taken as the most suitable model. Then, the current health state is recognized by estimating the state transition sequence. At last, the remaining useful life is estimated by integrating the mean of Gaussian distribution for each of the remaining state, and the lower limit.

In sum, HMM and modified HMM have been proactively explored in the past two decade or so in diagnostics and prognostics due to the natural convenient structure of HMM. For diagnostics, it seems to be a common practice to train an HMM for each health condition, and identify the monitored health condition by evaluating the probability. This field is heading from supervised training to unsupervised, autonomous clustering. This is done by training an HMM using

only healthy condition data, and identify novel condition by setting up a limit in observation conditional probability. For prognostics, the duration of HMM hidden state is usually taken as the prediction parameter. However, the exponential nature of discrete hidden states might not be suitable for most practical process. Instead, Gaussian distribution or modified duration in HSMM was proved to have better performance. For construction of unified framework that includes both diagnostics and prognostics, loglikelihood can serve as a similarity measure between the trained condition and the monitored one; and identifying the current state based on the chosen model, and estimating the RUL is then carried out.

### 2.3.2.Hidden Markov Model for Anomaly Detection and Process Monitoring

HMM has been applied for single component or system fault detection and process monitoring very actively in recent years. Unlike for diagnostics and prognostics, there is no absolute routine of how HMM should be implemented. Some of the efforts are review as follows, with some comments at the end.

For component-level anomaly detection, there are a few articles reviewed as follows:

Smyth (1994) proposed a method to use finite-state HMM as a complement to the existing pattern recognition system to improve the performance. As the title of the research article suggests, the approach for fault detection is part of a bigger control system to “smooth the classification decisions over time”. After the current state probability is obtained through either neural networks or Gaussian model, HMM will combine the previous sate probability estimates and estimate the current state based on the trained structure from the faulty library. The proposed

method took advantage of available domain knowledge like mean-time-to-failure to set the initial estimation of the transition matrix. Besides, it assumes that the faults are discrete and there is a fault library available.

Brown et al (2007) proposed a fault detection method for transformer condition monitoring. The sensor signal model is built using healthy condition data during the training stage. When monitoring the plant on-line, the sequence likelihood is calculated for each observation. It is assumed that when the sequence length gets longer, the observation loglikelihood will steadily decrease. However, when there is an abnormal observation, the loglikelihood will either decrease faster or slower, making the loglikelihood curve deviate from the normal condition ones. Only when the deviation occurs over several consecutive data points will it be taken as an anomaly. The number of data points that are deviating from the mean are counted to analyze sensitivity and to avoid false alarms.

For process monitoring, even more works are found and they are reviewed in the following paragraphs:

Yu (2010) proposed a process monitoring method based on HMM with an index combing global and local information. Model training is realized by applying PCA or ICA on the processed data, and then construct healthy state HMM. System monitoring is realized by inputting the monitor data into the constructed HMM and calculate the health index which is the combination of negative log-likelihood probability (NLL), which is a measure of global similarity between the observation and the model, and the Mahalanobis distance (MD) between the

current observation and the distribution it belongs to in the corresponding state, which is a measure of similarity between the observation and the local state.

Rashid and Yu (2012) proposed a process monitoring method by combining independent component analysis (ICA) with HMM. At the training scheme, the healthy condition data with different operating modes are classified by Viterbi algorithm. Then local ICA models are built based on each operating regime. During monitoring, the operating regime is first to be identified by HMM's maximum likelihood, and then ICA statistics are calculated and compared with local health limits to determine whether the system is in a faulty condition. However, when anomaly exists, the operating regime estimates might not be as accurate as it is in the healthy condition. This also poses potential risks of generating noisy health index and false alarms.

Cholette et al (2013) proposed a unified methodology covering topics of both condition monitoring and operation decision making. For condition monitoring, a complex HMM that is able to model different regime condition while depicting chamber degradation for semiconductor tools are constructed. Each operating regime or recipe is trained with an HMM with a certain degradation dynamic, and then the next operating regime's initial state is the ending state of the previous model with another degradation mechanism. After the model structure is defined and trained, a normalized loglikelihood slope is taken as the monitoring index, and whenever there is a drop, it means that an anomaly has been detected. The chosen HMM type is discrete multidimensional HMM, and during regime switch and model search, a genetic algorithm is applied for efficiency.

Ning et al (2014) brought up an interesting scheme to cope with process operating mode variety. The monitored system is modeled with two methods: one is HMM based statistics pattern analysis, the other is statistics pattern analysis mixture model. During monitoring period, the mode vector, which is the state transition sequence, is obtained by Viterbi algorithm, then the differential mode vector is calculated. The appropriate monitoring index is switched to automatically by comparing the differential mode vector value to the norm of it. The method is validated on a continuous stirred tank heater process and the Tennessee Eastman process.

All in all, HMM for anomaly detection and process monitoring has several common usage practices: 1) process monitoring based on HMM is usually modeled in an unsupervised fashion, meaning only healthy condition data are needed; 2) log-likelihood values and their changing behaviors can be used as a measure of similarity between the current observation and the normal condition; 3) process drift and multiple operating modes are one of the challenges to build an effective process monitoring model.

## 2.4. Research Gaps

1. Current challenges in data-driven process monitoring methods: lack of robustness.

   Data-driven models are signal models. Signal models are built over the historical pattern of data, and if the pattern changes over time, models will no longer be able to give correct estimation of the signals. That is why health monitoring performances usually suffer from the influence of

process drifts and non-stationary noises. Even though current process monitoring methods are trending towards more robust and adaptive schemes, model robustness in an environment of multiple operating modes with the presence of process drift is still a remaining issue. Existing robustness or adaptive ability are equipped by re-estimation (e.g. recursive PCA) or structure modification (e.g. sequential clustering using HMM) for unsupervised learning and novelty detection. Supervised learning methods usually have better performances, yet it always requires a more comprehensive library of operating condition data.

2. Why HMM is a good fit for health monitoring

First, since HMM can be used for almost all kinds of PHM applications: diagnostics, prognostics, and health monitoring, it is convenient to build a unified PHM model using HMM. The unified structure will reduce repetitive data preprocessing for different algorithms and increase model efficiency.

Second, HMM is a generative probabilistic model. The doubly embedded structure gives the model more flexibility and space for modification so that the most appropriate structure can be used for characterizing the monitored system. Also, since the model is evaluating a sequence of values instead of mapping the relationship one value by another, when a value is abruptly changing but the majority of the sequence still obeys the distribution in current state, the state estimation from HMM will not be influenced. If we assume that fault is considered to

be distribution shift, then there will be less false alarms compared to other methods.

Moreover, the model output, no matter what metric is used usually will correspond to a clear probability meaning. The structure of hidden state governing observation also coincides with most natural processes, and the state transition sequence estimation will usually give insight about the actual operating state of the system. Besides, prior domain knowledge such as mean-time-to-failure can be incorporated into the model and improve model performance.

# Chapter 3. Hidden Markov Model-Based Robust Health Monitoring

## 3.1. Introduction

Hidden Markov model (HMM) possesses the characteristic of modeling a doubly embedded stochastic process such that the unobservable governing states can be inferred from the distribution of observations. Industrial processes, like human speeches, the observed signals do not immediately reveal the states that are affecting machine behaviors. In PHM, HMM can be applied in diagnostics, prognostics, and fault detection. For diagnostics, several HMMs are trained with each representing a failure mode. Then the monitored asset condition is determined by comparing the conditional probability of observed signals given different models. For tool prognostics, the states can be taken as different severity levels of gradually developed faults. Usually full life span data are needed to train the model, and the remaining useful life can be inferred by estimating the current state and the duration of next transitions until failure. For fault detection, only normal condition data are needed, and the states can be modeled as different operating regimes, under the assumption that such regimes are discrete. This chapter will focus on introducing the theoretical preliminaries needed for the application in process monitoring using a discrete HMM.

This chapter is organized as follows: Section 3.2. will state the mathematical description of process monitoring problem; Section 3.3. will formulated the

overall methodology of the proposed health monitoring approach; Section3.4. explains in detail the techniques used for vector quantization; Section 3.5. elaborates on the topic of vector quantization; Section 3.6. deals with the construction of HMM, and how multi-regime modeling can be designed; and finally Section 3.7. will discuss about how to choose an appropriate health index for statistical process control.

## 3.2. Problem Statement

Consider an industrial process with a series of sensors collecting data from the machine under different operations. The collected data form a matrix consisting of $N$ variables as columns and samples as rows. Assuming a complete set of operational process lasts for time T, the variable matrix can be represented by

$$V = [v_1, v_2, \dots, v_N]_{T \times N}$$

where $v_i$ is a column vector representing the $ith$ variable. Typical operational process variables can be temperature, chemical release control signals, and machine mechanical component operation signals. As the overall process continues, same set of operations will be repeated and similar matrices will be obtained. The sequence length of each set is not necessarily the same. Therefore, a set of $M$ matrices are obtained represented by $\mathbb{V}$:

$$\mathbb{V} = \{V_1, V_2, \dots, V_M\}$$

Upon modeling, first, feature extraction will be performed on each variable matrix, obtaining a series of $K$ features, represented by

$$F = [f_1, f_2, \dots, f_K]_{M \times K}$$

Usually, there will be a discrete indicator variable for different machine operations. Typical features include summary statistics for time-series, e.g. mean, standard deviation, etc. For the same feature types, different operation might vary a lot. Usually, there will be a discrete indicator variable for different operations and features should be extracted based on different operating conditions. Once the feature matrices are obtained, the goal is to transform this feature matrix into a single health index with health limits, so that the condition of the machine can be determined even if the condition of the machine is drifting.

This performance drift of machines is one of the biggest challenges that need to be addressed during process monitoring. Unlike operating condition changes, there are no variables directly representing tool drift. It is not a variable that can be measured by sensors. Instead, the drift is “hidden” and is usually reflected in the process variables by changing their correlations or giving them offsets. Therefore, it is possible to infer the hidden drift from observed sensor signals. Due to the fact that tool performance drift can be caused by various reasons and scenarios vary a lot for sensor data behavior, the following assumptions are made for this research study:

1. Only machine healthy condition data are available.

This is a reasonable assumption due to the fact that users will usually know when the machine is running at an acceptable level, and whenever a fault happened, the machine will be shut down and inspection and maintenance will be conducted. As a result, machine nominal condition data will be abundant, while failure condition data will not be as sufficient. The term “nominal” and

“healthy” will be used interchangeably in this thesis and both of them refer to the machine condition when it is running well.

2. Faulty conditions will change the process variables in a different way from process drift.

Process drift will change the structure of process variables in terms of offsets and correlations. Faulty condition will also influence the process variables. However, the impact maybe more related to the shape of the signals. For example, nominal condition rolling-element bearing will usually have dominant frequency domain features in the lower frequency range, around rotational speed, while the energy at high frequency spectrum is usually low. Different speed and load will give different locations of dominant features in the spectrum, but it is still in the low frequency range close to rotational speed. Yet faulty rolling-element bearing will show high frequency spikes in the vibration signal so that in frequency spectrum, there will be higher energy in the high frequency range. Different operating conditions will affect the energy level of the spikes, but the spikes will still be in high frequency ranges.

3. There are a limited number of drifting scenarios occurring repetitively and the complete set of them in machine nominal state is available for modeling.

Causes of process drift in a complex system can be complicated. However, in some cases, the behaviors of the machines influenced by process drifts can be repetitive and the data related to such drifts will be accumulated over time. The model proposed in this thesis will be able to “learn” the pattern of these drifts,

and recognize them once they appear again. This assumption is needed since the model is not able to deal with novel drifting condition yet, and any novel drifts might confuse the model and make it difficult to tell whether it is a novel condition or a failure.

### 3.3. Methodology Overview

The overall methodology of the proposed approach is described in the flowchart shown in Figure 3-1. During training stage, only nominal condition data are needed from the monitored system. Before extracting any features, it is necessary to perform preprocessing to the data. Data preprocessing will make sure that the ratio of information related to concerned issues to the noise and disturbances are maximized.

After data preprocessing, feature extraction will be performed to represent the raw data with subtracted characteristics. Features can be time-domain summary statistics, frequency domain amplitudes, or time frequency domain properties such as wavelet-energy or short-time-Fourier-transform peaks. The extracted features will carry machine diagnostic information and the resulting feature matrix is then prognostic ready.

The kind of HMM used in this research is discrete hidden Markov model, which requires that the sequences input to the model should be discrete integers. Nonetheless, the raw data collected are usually continuous, or hybrid of continuous and discrete data type. Therefore, vector quantization will be carried out to cluster the signal and represent continuous signals with sequences of discrete integers.

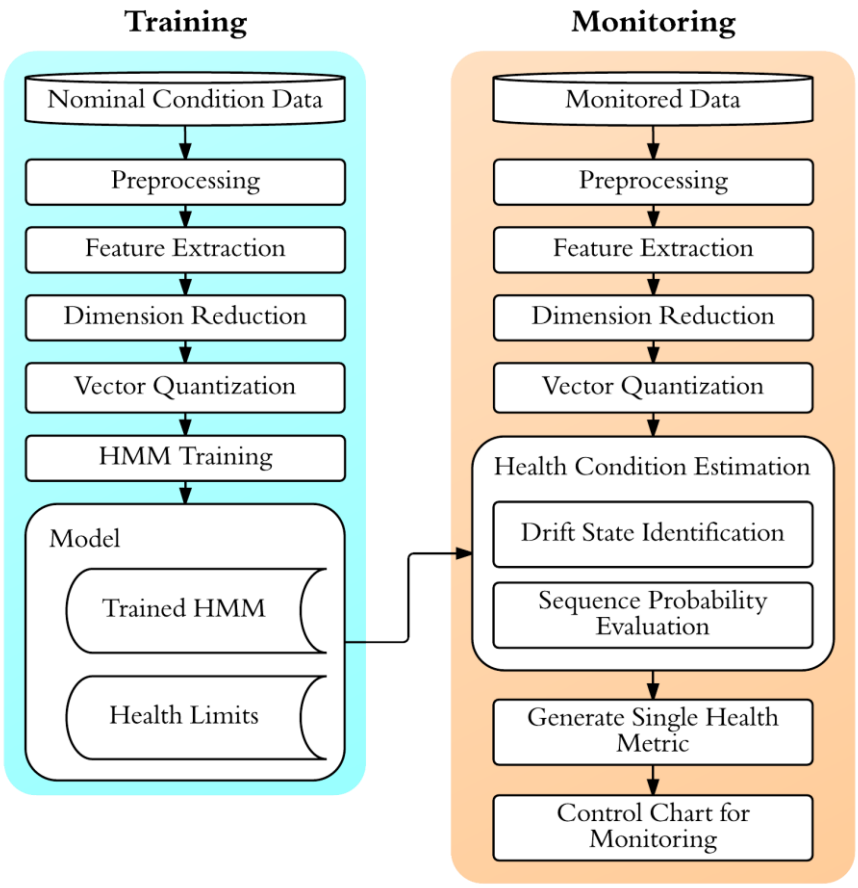



*Figure 3-1 Overall flowchart of the proposed robust process monitoring approach*

Dimension reduction is a necessary step. After vector quantization, there might be quite a few cluster or label numbers. If the number of features is also large, the computational efficiency will be low and it might be possible that there will be too few data points in certain clusters and it will lead to negative infinity in model log-likelihood. Standard dimension reduction methods include principal component analysis (PCA), or feature selection methods such as fisher ranking or gain ratio. Dimension reduction should be performed before vector quantization.

HMM will then be trained using the dimension-reduced and quantized features using Baum-Welch algorithm. Note that the training data in this research does not require the labeling of different drifting conditions. Basically, HMM states corresponds to different drift states of the system. If we assume that there is a limited number of drifting states, the number of drifting states, or regimes, will be estimated by evaluating he Bayesian Information Criterion (BIC) of the model for different number of states. After training, the trained HMM will automatically identify different regimes. Health limits should be calculated based on the deviation of the negative log-likelihood slope.

During monitoring stage, raw data will first go through the same steps such that the resulting features are preprocessed, dimension-reduced, and quantized. The drifting regime will then be identified by inferring the current state using Viterbi algorithm, and the system health condition will be evaluated by calculating the normalized negative log-likelihood slope of the monitored observations and comparing with the health limits.

The critical procedures and the theories will be elaborated in the following sections.

## 3.4. Feature Extraction and Dimension Reduction

### 3.4.1.Preprocessing

Owing to sensor malfunction or abnormality in signal transmission, there are all kinds of defects in the raw data that need to be addressed so that the errors would not mislead the analysis result. Data quality evaluation is an independent

topic itself (Chen, 2012). In general, the purpose of preprocessing is to ensure that the data being dealt with are physically correct and statistically acceptable. Inspection of the physical aspect of the data is usually reflected in vibration signals, since vibration signals are widely used in various industrial applications such as wind turbine monitoring. Automatic validation of vibration signals are designed (Jabłoński, Barszcz and Bielecka, 2011; Jablonski & Barszcz, 2013) based on a series of statistical measurements to evaluate the physical senses of the signal. The other perspective is purely from statistics. For process monitoring, domain knowledge of every monitored parameter is not always available. Therefore preprocessing will be performed by eliminating missing values, outliers, and repetitive data points that are sharing the same timestamp. Outlier detection is of particular interest in certain cases and the method selection can be subjective. The common detection routine is to set a threshold as big as five times the standard deviation, and remove the data points that are outside this range; another method is to perform Grubbs test to detect outliers (Grubbs, 1969). In summary, data preprocessing is of vital importance. It eliminates the noise and disturbances caused by unwanted sources or errors, and increases the portion of the data that are truly related to the property to be investigated.

### 3.4.2.Feature extraction

After data preprocessing, compact information is usually to be extracted from the preprocessed raw data to represent their characteristics. This kind of mathematical form of representation is referred to as features. Mathematically, a feature is a vector containing a limited number of elements. Features can be

extracted on the basis of domain knowledge, advanced signal processing, or simple summary statistics. Generally speaking, features can be categorized into three basic classes: time-domain features, frequency-domain features, and time-frequency domain features.

Time-domain features can be (1) basic statistics like mean, variance, or kurtosis, (2) distributions like histogram or Gaussian parameters, and (3) time measurements such as time-delay. When thorough knowledge about the system is available, application-specific features can also be extracted.

Frequency-domain features mainly refer to Fast Fourier Transform (FFT)-based frequency spectrum measurements. For stationary signals, FFT can be adopted to extract amplitudes at interested frequencies; sideband amplitudes around certain frequencies; or entropy differences between different spectrums. If the sampling rate is high enough, envelope analysis can be performed to increase the signal-to-noise ratio for extracting diagnostic information. Frequency domain feature extraction usually requires significant knowledge in signal processing, and a whole family of methods are available and being developed for noise reduction and diagnostic portion enhancement. It is usually widely used in rotary machinery diagnostics.

Time-frequency domain feature extraction becomes more useful than frequency domain methods when the signal is non-stationary, i.e. the frequency spectrum is time-variant. Typical time-frequency methods include wavelet decomposition and short-time Fourier transform (STFT). Basically, a window size is selected to perform frequency domain analysis and a visualization form is

adopted to identify significant changes in frequency domain at certain time-transients. However, usually these methods are computationally expensive, so they are preferred when the sampling rate is high and the monitored plant requires high reliability and readiness.

The desired properties of features are:

(1) Containing diagnostic information than is indicative of the targeted problem;

(2) Contain the least amount of redundancy.

The first properties can usually be realized if the feature extraction process is performed appropriately. However, it does not guarantee the resulting features are all “good” and there might be a lot of correlated features. This requires the next step – feature selection and dimension reduction, to eliminate redundant and correlated features, and only keep the ones that are most helpful for problem detection.

### 3.4.3.Dimension reduction

Dimension reduction is the process of reducing feature matrix dimension so that the processed features are less relevant to each other and the diagnostic information contained is more pronounced. The purpose of dimension reduction is to improve algorithm accuracy, speed up the learning process, and help reduce the number of sensors needed. There are three categories of dimension reduction. One is the reduction of instances, which can be realized by down-sampling or clustering. Second is variable selection. Features that can best

differentiate different classes (e.g. healthy & different failure modes) will be selected based on certain criteria. Popular methods include Fisher score and information gain ratio. The third category includes principal component analysis (PCA) or independent component analysis (ICA). These methods will project the high dimensional feature matrix into lower dimensions, and reduce the redundancy in the lower dimensional space. The number of principal components selected can be determined by scree plot, Kaiser One method, or cumulative variation proportion methods. Detailed explanation of PCA can be found in Section 2.2.1.

The choice of dimension reduction methods depends on the problems and data availability. Instance reduction is used when faced with data collected with a sampling rate higher than enough. Down-sampling the data will speed up the analysis process. Variable selection through Fisher discriminant criteria or information gain is suitable for cases when faulty data are available. In process monitoring, only nominal condition data are provided for algorithm training. Therefore, PCA is chosen as the appropriate dimension reduction tool for the method proposed in this thesis.

## 3.5. Vector Quantization

Vector quantization is the process that maps the continuous/discrete data into a finite number of code words so that data compression is realized (Gray, 1984). Image compression is one of its primary applications. There are generally two classes of vector quantizers. One is memoryless vector quantizers, which quantize the data without any adaptive ability to adjust the codebook; the other

vector quantizers with memory, which utilizes the codebooks in the past – memory, to adjust the quantization. An important metric for the “goodness of quantization” is distortion. It can be as simple as the ubiquitous mean squared error (MSE), or complicated and yet more general measurement like Itakura-Saito distortion. Vector quantizers with memory do not outperform memoryless vector quantizers in terms of minimizing distortion, but it has the advantage of reducing computational complexity (Gray, 1984). In this research study, vector quantizers will not be dealing with a large amount of data samples for online usage. Therefore, the more straightforward memoryless vector quantizers are adopted.

Mathematically, the quantization process for memoryless vector quantizers involves two mappings (Gray, 1984):

1. The mapping from input vector $X$ to a channel symbol set $M$ serving as encoder $\lambda$;
2. The mapping from each symbol $\alpha$ to a reproduction alphabet $\hat{A}$.

In this research study, in order to discretize the continuous feature vectors, each code word is mapped to discrete integers. The process of vector quantization is illustrated in Figure 3-2.

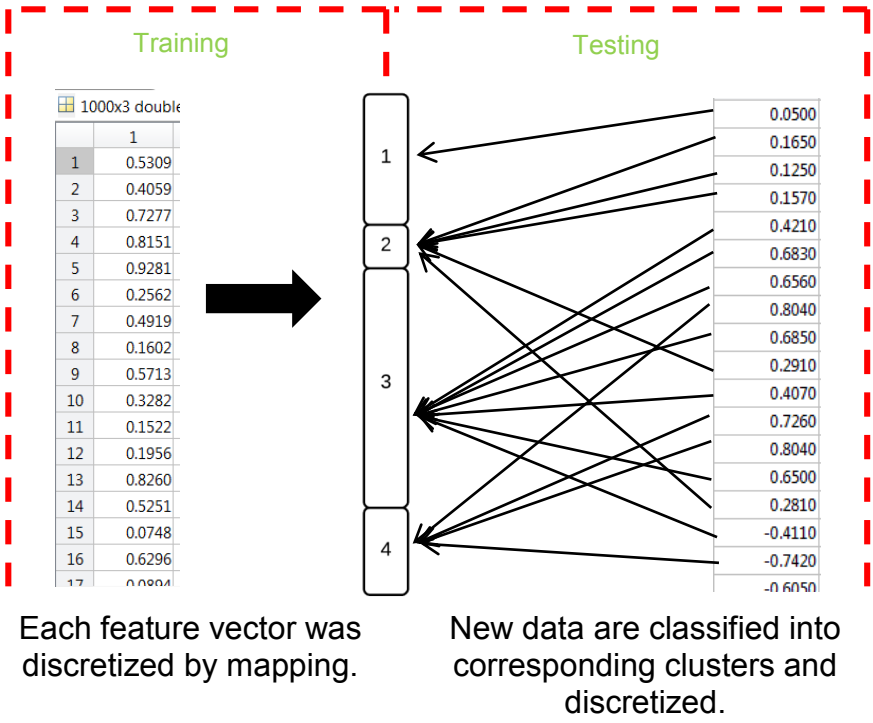



*Figure 3-2 Illustration of vector quantization for continuous value discretization*

Common vector quantization methods include Linde–Buzo–Gray (LBG) algorithm (Gray, 1984; Yang, Xu and Chen, 1994), which is similar to k-means clustering, linear predictive coding (LPC) (Yang & Xu, 1994), SOM-based methods (Somervuo & Kohonen, 1999; Infantesb & Longa, 2004; Liu, 2008; Cholette, 2012). SOM is adopted in this research. Details of the description for SOM can be found in (Kohonen, 1990). Note that SOM is merely used as a clustering tool here for the quantization of each continuous feature vector; and for new vectors, SOM will serve as a classification tool to label the new vector with integers.

## 3.6. Hidden Markov Model for Process Drift Modeling

*Table 3-1 List of notations*

| Variable | Meaning |
|---|---|
| $S_i$ | The $ith$ state from $N$ distinctive states |
| $s_t$ | The state occurring at Time $t$ |
| $O_k$ | The $kth$ observation from $M$ distinctive observations |
| $O$ | $O = \{o_1, \dots, o_T\}$, represents a set of observations |
| $o_t$ | The observation occurring at Time $t$ |
| $\pi_i$ | Initial probability of State $S_i$ |
| $a_{ij}$ | The transition probability from State $S_i$ to State $S_j$ |
| $b_i(o_t)$ | The observation probability of $o_t$ generated by State $S_i$ at Time $t$ |
| $\lambda$ | A hidden Markov model |

### 3.6.1.Theoretical Background

Hidden Markov model (HMM) is a complex stochastic process that consists of a Markov and an observation model (Murphy, 2012). It assumes that the system follows an underlying Markov process which is NOT observable. The values that ARE observed are generated through another set of stochastic process. An observation probability density matrix links these two processes. This probability density matrix describes the distribution of observations under a certain state. The states transit from one to another following a transition probability matrix. And the state at this time is only dependent on ONE state occurring before it. The process is illustrated in Figure 3-3.

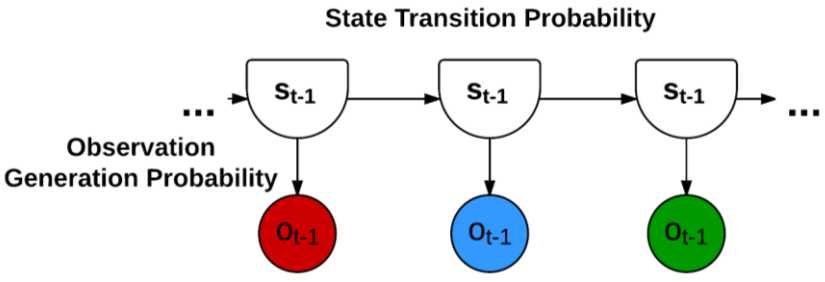


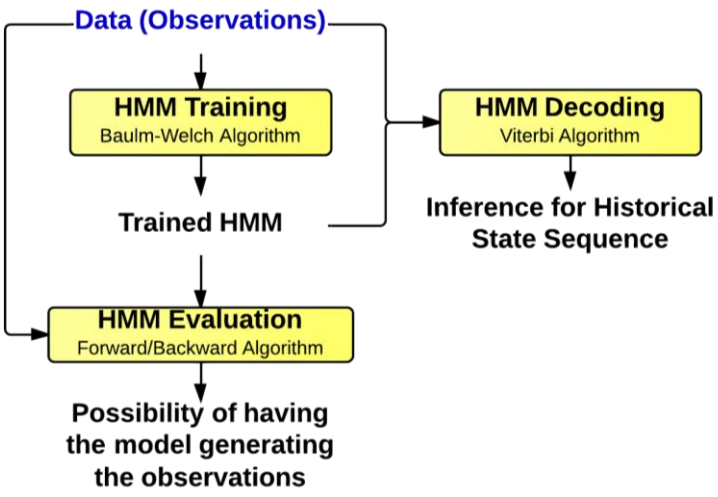

*Figure 3-3 Illustration of HMM process*

An HMM is consisted of three basic elements, which are

- Initial state distribution $\pi$,
- State transition matrix $A$, and
- Observation matrix $B$.

There are three essential problems related to HMM:

- Evaluate the possibility of the data being generated by the trained model (HMM Evaluation)
- Infer the state sequence behind the observations (HMM Decoding)
- Find out the stochastic pattern (model) that generates this sequence (HMM Training)




- *Figure 3-4 Three essential problems of HMM*

The input of all three questions is a sequence of observations, and an HMM model. The output and the relations for each of the problem are summarized in Figure 3-4. The next few subsections will introduce each of the three problems

and explain the classical mathematical algorithms that can efficiently solve these problems.

#### 3.6.1.1. HMM Evaluation

The evaluation of a sequence of observation is the process of calculating the probability of observing the given sequence under the HMM model. Mathematically, if we represent the observation sequence with $O$, and the HMM model to be $\lambda$, then the evaluation problem is trying to obtain: $P(O|\lambda)$. By definition, the evaluation probability can be obtained by calculating the following equation:

$$P(O|\lambda) = \sum_{i}^{N} \pi_{s_1} b_{s_2}(o_1)\, a_{s_1 s_2} b_{s_2}(o_2) \dots a_{s_t s_{t+1}} b_{s_{t+1}}(o_{t+1}) \dots a_{s_{T-1} s_T} b_{s_T}(o_T) \quad (1)$$

where the meaning of the symbols can be found in Table 3-1. However, the computational complexity of this equation increases dramatically as the number of observations gets larger. Therefore, instead of calculating the equation directly, we can break it down to each timestamp, and find out the recursively occurring terms for efficient computation. For representation convenience, the model is set to be a prerequisite, so that $P(O|\lambda) = P(O)$, given $\lambda$.

At $t = 1$,

$$P(o_1) = P(\mathrm{o_1}|\mathrm{s_1} = \mathrm{S_1})\mathrm{P}(\mathrm{s_1} = \mathrm{S_1}) + \mathrm{P}(\mathrm{o_1}|\mathrm{s_1} = \mathrm{S_2}) + \cdots + \mathrm{P}(\mathrm{o_1}|\mathrm{s_1} = \mathrm{S_N})$$

$$= \sum_{i=1}^{N} b_i(o_1)\pi_i$$

meaning at t=1, the total probability of observing $o_1$ equals to the sum of each state generating $o_1$ multiplied by that particular state occurring. We introduce a new term here to represent that given Model $\lambda$, the probability of observing $o_1, \dots, o_t$ from Time 1 to Time $t$, and the state at Time $t$ being $S_i$ is $\alpha_t(i)$.

$$\alpha_t(i) = P(o_1, \dots o_t, s_t = S_i|\lambda)$$

Therefore, $P(o_1) = \sum_{i=1}^{N} \alpha_1(i)$. This is illustrated by the following graph.

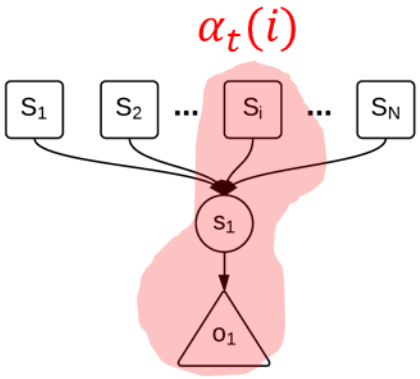



*Figure 3-5 Illustration of $\alpha_t(i)$ in the forward algorithm*

Continuing the calculation at $t = 2$, we have

**The probability of observing $o_1, o_2$ from Time 1 to Time 2, and the state at Time 2 being $S_j$ is $\alpha_2(j)$.**

$$P(o_1, o_2) = P(o_1)P(o_2|o_1)$$

$$= P(o_2|s_2 = S_1)P(s_2 = S_1|s_1 = S_1)P(s_1 = S_1)P(o_1|s_1 = S_1) + \cdots +$$

$$P(o_2|s_2 = S_j)P(s_2 = S_j|s_1 = S_i)P(s_1 = S_i)P(o_1|s_1 = S_i) + \cdots +$$

$$P(o_2|s_2 = S_N)P(s_2 = S_N|s_1 = S_N)P(s_1 = S_N)P(o_1|s_1 = S_N)]$$

$$= \sum_{j=1}^{N} \left[ \sum_{i=1}^{N} P(o_2|s_2 = S_j)P(s_2 = S_j|s_1 = S_i)P(s_1 = S_i)P(o_1|s_1 = S_i) \right]$$

$$= \sum_{j=1}^{N} b_j(o_2) \left[ \sum_{i=1}^{N} a_{ij}\pi_i \, b_i(o_1) \right] = \sum_{j=1}^{N} b_j(o_2)[\sum_{i=1}^{N} a_{ij}\alpha_1(i)]$$

$$= \sum_{j=1}^{N} \alpha_2(j)$$

This equation can be illustrated as what is shown in Figure 3-6.

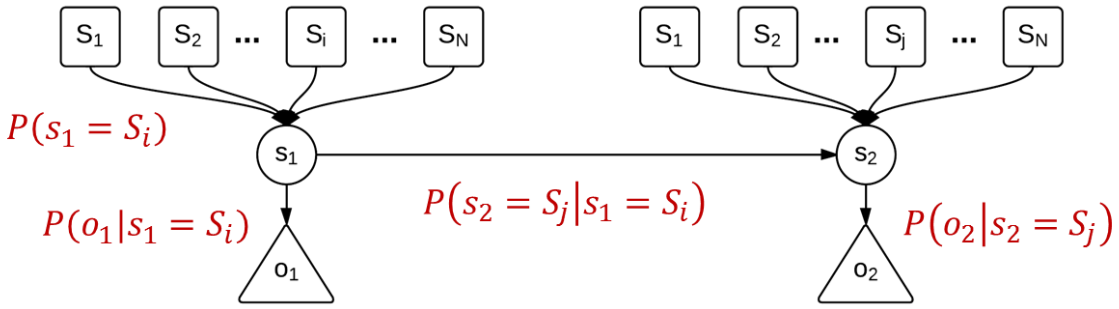



*Figure 3-6 Illustration of forward algorithm at Time 2*

Based on the above calculation, the probability of $P(□ = \{O_1, \dots, O_T\}$ can be easily represented by $P(O = \{o_1, o_2, \dots, o_T\}) = \sum_{i=1}^{N} \alpha_T(i)$.

Note that

$$\alpha_2(j) = b_j(o_2)\left[\sum_{i=1}^{N} a_{ij}\alpha_1(i)\right]$$

Thus, we can recursively calculate

$$\alpha_t(j) = b_j(o_2)[\sum_{i=1}^{N} a_{ij}\alpha_{t-1}(i)]$$

This algorithm is called the forward algorithm. It is a type of dynamic programming (Bellman, 1956). Therefore, we can summarize the algorithm in the following algorithm description.

*Table 3-2 Summary of forward algorithm*

| Forward Algorithm: |
|---|
| • Define $\alpha_t(i) = P(o_1, \dots, o_t,\ s_t = S_i \vert \lambda)$ |
| 1) Initialization $\alpha_1(i) = \pi_i b_i(o_1),\ i = 1, \dots, N$ |
| 2) Induction $\alpha_t(j) = \left[\sum_i^N \alpha_{t-1}(i) a_{ij}\right] b_j(o_t),\ j = 1, \dots, N; t = 2, \dots, T$ |
| 3) Termination $P(O \vert \lambda) = \sum_i^N \alpha_T(i)$ |

Forward algorithm is calculating the sequence probability from Time 1 to Time T, as shown in Figure 3-7. Equivalently, if the calculation is done from the last timestamp to the first one, similar recursive calculation can be conducted with a different format. This is referred to as the backward algorithm, which is summarized in Table 3-2.

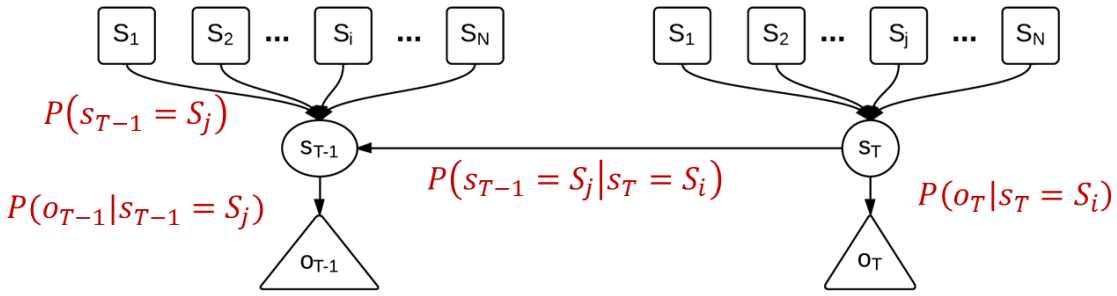



*Figure 3-7 Illustration of backward algorithm*

*Table 3-3 Summary of backward algorithm*

| Backward algorithm |
|---|
| • Define $\beta_t(i) = P(o_{t+1}, o_{t+2}, \dots, o_T,\ s_t = S_i \vert \lambda)$ |
| 1) Initialization $\beta_T(i) = 1,\ i = 1, \dots, N$ |
| 2) Induction $\beta_t(i) = \sum_{j=1}^N a_{ij} b_j(o_{t+1}) \beta_{t+1}(j), j = 1, \dots, N; t = T - 1, T - 2, \dots, 1$ |
| 3) Termination $P(O \vert \lambda) = \sum_i^N \beta_T(i)$ |

The computation complexity of both forward and backward algorithms are on the order of $N^2T$ (Rabiner, 1989), and the structure of calculating them is the efficient lattice structure.

#### 3.6.1.2. HMM Decoding

HMM decoding is the inference of hidden state sequence from observations. Mathematically, the problem is: given a sequence of observations $o = \{o_1, \dots, o_T\}$ over time $t = 1, \dots, T$, plus an HMM $\lambda$, what is the most probable state sequence?

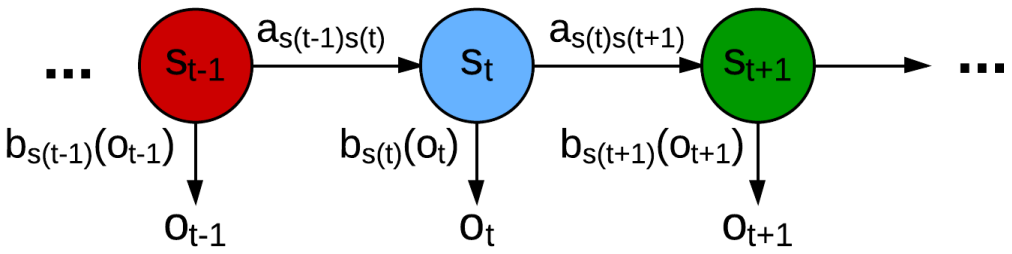



*Figure 3-8 State sequence estimation from observation given the model*

As shown in Section 3.6.1.1. , the key is to find an efficient way of calculating the sequence probability. The most popular decoding algorithm is the Viterbi algorithm. The basic idea is to run through every state at each timestamp, and record the state at each step that maximizes the overall highest probability. More specifically, it is realized by the following procedures (Viterbi, 1967).

- Define $\delta_t(i) = \max_{s_1,\dots,s_{t-1}} P(s_1, \dots, s_{t-1}, s_t = S_i, o_1, \dots, o_t|\lambda)$ as the highest probability for a state sequence from Time 1 to Time t.
- 3 Steps – an optimization consideration on forward algorithm
  - **1. Recursively compute the overall highest possible probability**

- t=1, $\delta_1(i) = P(s_1 = S_i, o_1|\lambda) = \pi_i b_i(o_1)$ (Initialization)
- t=2, $\delta_2(j) = \max_{1\le i\le N} P(s_1 = S_i, s_2 = S_j,\ o_1, o_2|\lambda)$

  $= \max_{1\le i\le N} P(o_2|s_2 = S_j)P(s_2 = S_j|s_1 = S_i)P(s_1 = S_i)P(o_1|s_1 = S_i)$

  $= \max_{1\le i\le N} b_j(o_2)a_{ij}\pi_i b_i(o_1) = \max_{1\le i\le N} \delta_1(i)$
- …, $\delta_t(j) = \max_{1\le i\le N} b_j(o_t)\delta_{t-1}(i)$

– **2. Record the state $S_i$** that maximizes $P(s_1, \dots, s_{t-1}, s_t = S_i, o_1, \dots, o_t|\lambda)$ at each iteration

– **3. Back-track and get the state sequence**

The algorithm is summarized in the following table (Rabiner, 1989).

*Table 4 Summary of Viterbi algorithm*

Viterbi algorithm

- Define $\delta_t(i) = \max_{s_1,\dots,s_{t-1}} P(s_1, \dots, s_{t-1}, s_t = S_i, o_1, \dots, o_t|\lambda)$ as the highest probability for a state sequence.

1) Initialization

$$\delta_1(i) = \pi_i b_i(o_1),\ i = 1, \dots, N$$

Keep track of the state: $\phi_1(i) = 0$

2) Recursion

$$\delta_t(j) = \max_{1\le i\le N}\left[\delta_{t-1}(i)a_{ij}\right]b_j(o_t),\ t = 2, \dots,\ T; j = 1, \dots,\ N$$

$$\phi_t(j) = \operatorname*{argmax}_{1\le i\le N}[\delta_{t-1}(i)aij]$$

3) Termination

$$P^* = \max_{1\le i\le N} \delta_T(i)$$

$$s_T^* = \operatorname*{argmax}_{1\le i\le N} \delta_T(i)$$

4) Path backtracking

$$s_t^* = \phi_{t+1}(s_{t+1}^*),\ t = T-1,\ T-2, \dots,1$$

#### 3.6.1.3. HMM Learning

HMM learning is the process of estimating the parameters of the model, $\theta = [\pi, A, B]$ using the observation and the initial guess of the parameters. Mathematically, the problem can be stated as: given a sequence of observation $o = \{o_1, \dots, o_T\}$ over time $t = 1, \dots, T$, and an initialized model

$$\pi = \{\pi_i\} = \{P[s_1 = S_i]\}$$

$$A = \{a_{ij}\} = \{P[s_{t+1} = S_j | s_t = S_i]\}$$

$$B = \{b_j(k)\} = \{P[o_t = O_k | s_t = S_j]\}$$

re-estimate $\theta = [\pi, A, B]$ so that $P(O|\lambda)$ is maximized.

The most classic algorithm to solve the estimation problem is the Baum-Welch algorithm (Rabiner, 1989; Murphy, 2012). It is essentially an expectation-maximization (EM) algorithm to recursively estimate the three parameter matrices.

More specifically, the algorithm is carried out as follows:

- Define two variables
  - Let $\epsilon_t(i,j) = P(s_t = S_i, s_{t+1} = S_j | O, \lambda)$
    - the probability of transition from State $S_i$ at Time $t$, to State $S_j$ at Time $t+1$, namely, given the model and the observation.
  - Let $\gamma_t(i) = P(s_t = S_i | O, \lambda)$

- the probability of State $S_i$ being at Time $t$, given the model and the observation.

- **E-Step: Compute two expectations**
  - $\sum_{t=1}^{T} \epsilon_t(i,j)$ = expected number of transitions from $S_i$ to $S_j$
  - $\sum_{t=1}^{T} \gamma_t(i)$ = expected number of transitions from $S_i$ to any states
- **M-Step: Update three parameters**
  - $\overline{\pi_i} = expected\ occurence\ number\ of\ S_i\ at\ Time\ t = \sum_{t=1}^{1} \gamma_t(i) = \gamma_1(i)$
  - $\overline{a_{ij}} = \frac{expected\ number\ of\ transition\ from\ S_i to\ S_j}{expected\ number\ of\ transitoni\ from\ S_i\ to\ any\ states} = \frac{\sum_{t=1}^{T} \epsilon_t(i,j)}{\sum_{t=1}^{T} \gamma_t(i)}$

$$\overline{b_j(k)} = \frac{expected\ occurence\ number\ of\ S_j\ when\ observing\ O_k}{expected\ occurence\ number\ of\ S_j}$$

$$= \frac{\sum_{t=1,\ s.t.\ o_t=O_k}^{T} \gamma_t(i)}{\sum_{t=1}^{T} \gamma_t(i)}$$

With the calculation of forward and backward algorithm introduced earlier, $\epsilon_t(i,j)$ can be efficiently calculated as

$$\epsilon_t(i,j) = P\left(s_t = S_i, s_{t+1} = S_j \middle| O, \lambda\right) = \frac{P(s_t = S_i \cap s_{t+1} = S_j \cap O \cap \lambda)}{P(O \cap \lambda)}$$

$$= \frac{\frac{P(s_t = S_i \cap s_{t+1} = S_j \cap O \cap \lambda)}{P(\lambda)}}{\frac{P(O \cap \lambda)}{P(\lambda)}}$$

$$= \frac{P(s_t = S_i, s_{t+1} = S_j, O|\lambda)}{P(O|\lambda)} = \frac{\alpha_t(i)a_{ij}b_j(o_{t+1})\beta_{t+1}(j)}{\sum_i^N \sum_j^N \alpha_t(i)a_{ij}b_j(o_{t+1})\beta_{t+1}(j)}$$

And notice that the other term can be represented by $\epsilon_t(i,j)$,

$$\gamma_t(i) = \sum_j^N \epsilon_t(i,j)$$

Again, the algorithm is summarized in the table below.

*Table 5 Summary of Baum-Welch algorithm*

Baum-Welch algorithm

- Define $\epsilon_t(i,j) = P\left(s_t = S_i, s_{t+1} = S_j \middle| O, \lambda\right)$, and $\gamma_t(i) = P(s_t = S_i|O,\lambda)$.

1) Initialization
   - Randomly define $\pi_i$, $A$, and $B$
2) E-Step
   - Compute $\alpha_t$ and $\beta_t$ by following forward and backward algorithm
   - Compute $\epsilon$ and $\gamma$
3) M-Step
   - Update $\pi_i$, $A$, and $B$
4) Repeat Step 2 and 3 until $P(O|\lambda)$ stops increasing

Note that Baum-Welch can only get the local maximum. It is not able to obtain the global maximum. Therefore, the performance of estimation is very sensitive to initial values. This is why a lot of efforts have been made to give better initial guesses of the model parameters and ensure that the result is the global optimum (Juang & Rabiner, 1990; Larue, Jallon and Rivet, 2011; Clemente, Heckmann and Wrede, 2012). In this research study, in order to cope with this problem, the algorithm is trained 100 times with random initialization values, and

the model that has the highest probability (log-likelihood) is chosen as the optimal model.

### 3.6.2.Process Drift Modeling

Process drifts widely exist in any manufacturing or chemical process. The definition of process drift unexpected shift of a measured process parameter(s) and/or resulting product attributes away from its intended target over time. Modeling process drifts is important for process monitoring, because models constructed based on only one regime will oftentimes not work well in other regimes. In order to model process drift with HMM, first of all, a few assumptions are proposed:

1) there are a finite number of drifting states;
2) the drifting states are discrete;
3) current drifting states is only affected by the previous one state.

With these assumptions, the drifting states can be modeled as a Markov process. Mathematically, the drift process has the property that

$$P(S_k = s_k | S_{k-1} = s_{k-1}, S_{k-2} = s_{k-2}, \dots, S_0 = s_o) = P(S_k = s_k | S_k = s_k)$$

However, usually there are no parameters that are able to directly measuring whether any drift is happening or the degree of drifting. Yet process drifts will change average / variable correlation / variation of the sensor signals, even if the equipment is considered healthy (generating acceptable yield). If we assume that at each process drifting state, sensor signal readings follow a stationary probability distribution, and this distribution changes according to the drifting

state. This mechanism fits the scheme of HMM. The drifting states that follows a Markov process are the hidden states, sensor signals are the observations, as shown in Figure 3-9.

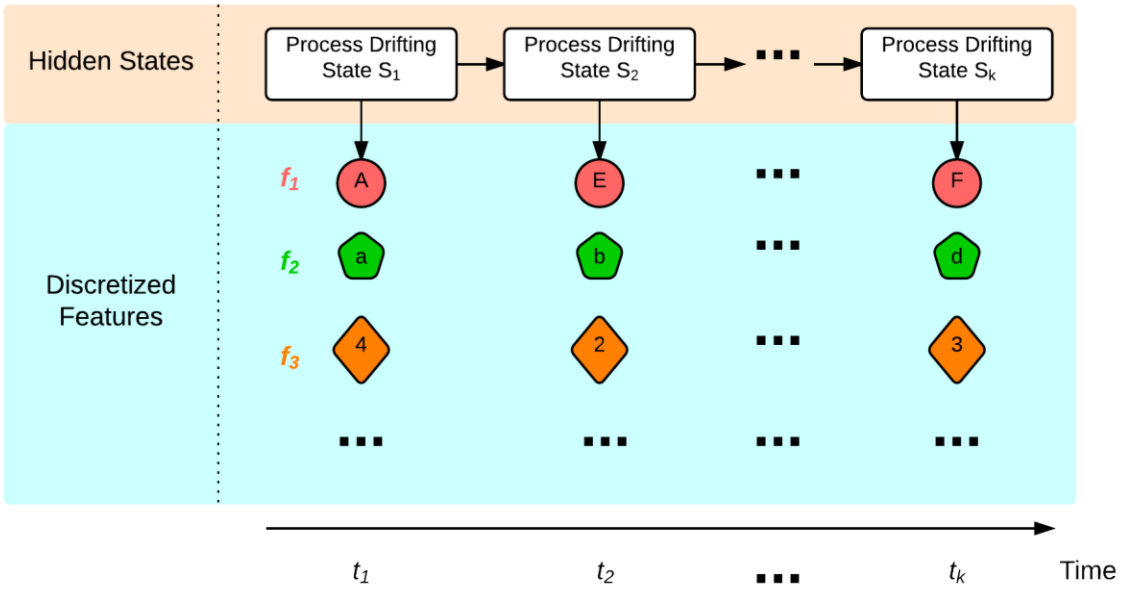



*Figure 3-9 Illustration of process drifting state modeling using HMM*

## 3.7. Health Metric

Health metric for process monitoring is an important topic. In this research study, the slope of the log-likelihood is taken as the process monitoring metric to detect anomalies. The rationale behind this is that if the observation agrees with the estimated model, then the log-likelihood will be steadily decreasing as the sequence of the observation gets longer. However, whenever there is an anomaly, the log-likelihood will deviate from the original curve, and a “drop” will occur in the slope since it will decrease faster (Brown et al., 2007; Cholette, 2012; Cholette et al., 2013).

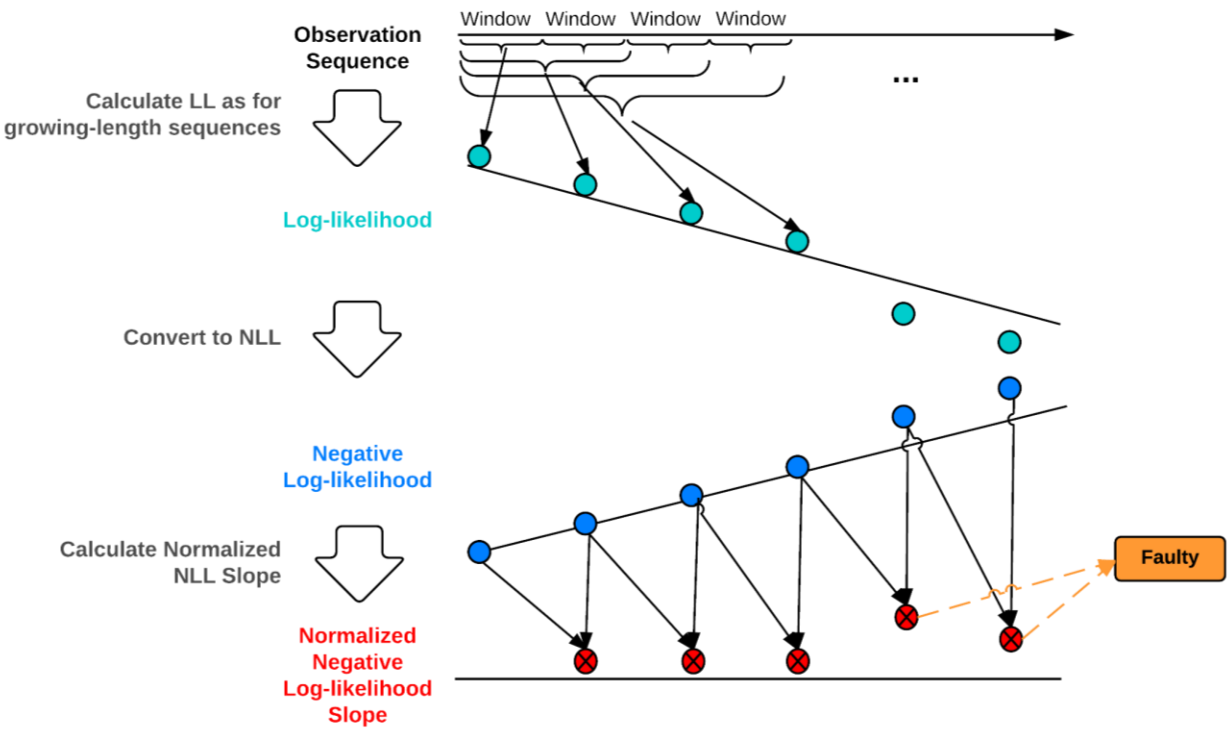



*Figure 3-10 Illustration of log-likelihood slope for process monitoring (MOD, change to NLL)*

This metric is further improved and scaled by evaluating the negative log-likelihood (NLL) slope and normalization. Mathematically, the metric is represented by:

$$s_{norm,k} = \frac{[(-LL_k) - (-LL_{k-1})] - \mu_s}{\sigma_s}$$

where $s_{norm,k}$ is the normalized negative log-likelihood slope at time $k$, $LL_k$ is the log-likelihood at time $k$, $\mu_s$ is the mean of the negative log-likelihood slope for training data, and $\sigma_s$ is the standard deviation of the negative log-likelihood slope for training data.

# Chapter 4. Case Study: Semiconductor Etching Process Monitoring

## 4.1. Introduction

The proposed method is going to be validated on an industrial application, and compared with benchmarking methods using PCA-MSPC and SOM-MQE. A public data set from Eigenvector Research Inc. are adopted (http://www.eigenvector.com/data/Etch/). Data are from a LAM 9600 Metal Etcher (hereafter referred to as "LAM data") over the course of etching 129 wafers during three separate experiments. Among all the experiments, in total there are 108 runs that are considered "normal", with yielded wafer of good quality; the rest 21 runs are carried out with induced faults. Each experiment contains a few runs of faults. A brief description of the data set can be found in Table 4-1. The three experiments were conducted separately during three different time periods, which leads to significant differences in data covariance and mean. Note that the faulty runs of were re-organized for each experiment so that the severity of the fault is increasing over time, in order to imitate a degradation process.

*Table 4-1 Data description wafer run health conditions for LAM data*

| Experiment Name | Number of Wafer Runs | |
|---|---|---|
| | Healthy | Faulty |
| #29 | 34 | 9 |
| #30 | 37 | 6 |
| #31 | 37 | 6 |
| **Total** | **108** | **21** |

This data set is chosen for validation of the proposed approach because:

1) It is a relatively ideal case where process drifts occur and greatly affect model performance;
2) No algorithm has achieved 100% of detection rate yet, even though a lot of published research studies used this data set.

In the following sections, data description and pre-processing procedures are explained first. Then features extracted from the raw data as the input for different algorithms are explained. Then the results from benchmarking methods are presented, followed by the result from the proposed method. At the end, discussion and benchmarking are formulated.

## 4.2. Data Pre-processing, Feature Extraction, and Dimension Reduction

A list of variables for the LAM data can be found in Table 4-2. In total, there are 21 variables. Sampling rate is approximately 1 Hz. “Time” represents timestamp, and “Step Number” is the label for operation steps. For this etching process, there are totally 6 process steps. However, in the data set provided, only the key steps Step 4 and 5 are included (Wise et al., 1999). The rest of the variables are the sensor signals that are usually called “trace signals”.

Before applying any process monitoring methods, data pre-processing is performed. Data pre-processing is the first step for building a data-driven model. The purpose of data pre-processing is to inspect data quality so that there are no missing values, outliers, or physically incorrect instances that will mislead the data analysis result.

*Table 4-2 List of variables in LAM data*

| # | **Variable Name** | **Remarks** |
|---|---|---|
| 1 | Time | Timestamp. |
| 2 | Step Number | Label for different operations. For LAM data, there are two operation steps: Step 4 and Step 5. |
| 3 | BCl3 Flow | Sensor signals from equipment monitoring. |
| 4 | Cl2 Flow | |
| 5 | RF Btm Pwr | |
| 6 | RF Btm Rfl Pwr | |
| 7 | Endpt A | |
| 8 | He Press | |
| 9 | Pressure | |
| 10 | RF Tuner | |
| 11 | RF Load | |
| 12 | RF Phase Err | |
| 13 | RF Pwr | |
| 14 | RF Impedance | |
| 15 | TCP Tuner | |
| 16 | TCP Phase Err | |
| 17 | TCP Impedance | |
| 18 | TCP Top Pwr | |
| 19 | TCP Rfl Pwr | |
| 20 | TCP Load | |
| 21 | Vat Valve | |

The convenience of available toolboxes and software has made data analysis result just a few lines of code or clicks away from raw data to summary. However, “garbage in, garbage out.” It is paramount that the data integrity is thoroughly inspected to avoid misleading information or conclusion being drawn from the analysis. The aforementioned purpose has already revealed some of the tasks of data pre-processing. Here they are summarized in the first row of Table 4-3. In the data set, there are 108 runs, including both normal condition and faulty condition data. Among them, Run 56 was eliminated due to insufficient instance number.

*Table 4-3 Summary of Data Pre-processing, Feature Extraction, and Dimension Reduction methods*

| **Step** | **Methods** |
|---|---|
| Data Pre-processing | 1. Eliminate missing data<br>2. Eliminate outliers<br>3. Eliminate runs without enough data instances<br>4. Eliminate values that are out of physically meaningful ranges |
| Feature Extraction | Summary statistics:<br>1. Mean<br>2. Standard deviation<br>3. Minimum<br>4. Maximum<br>5. Kurtosis<br>Note:<br>• Operation-specific: In total there are two steps of operations, Step 4 and 5. Features were extracted separately for each operation step. |
| Dimension Reduction | Principal component analysis |

After data pre-processing, summary statistics are extracted from time-series. Extracting summary statistics is a common practice for semiconductor trace signals. The extracted statistics are listed in Row 2 of Table 4-3. Basically, one feature is extracted every 5 data points for Step 4 and Step 5, respectively, because trace signals usually have different mean and variances for different operating steps, as shown in Figure 4-1. Separating features from the two different steps will facilitate better comparison between different runs.

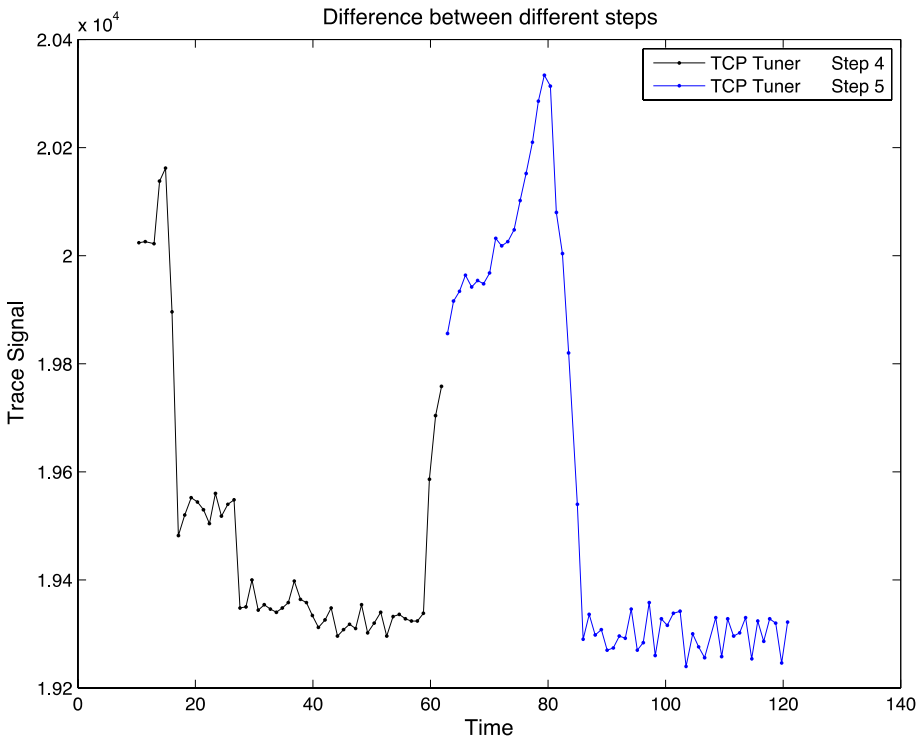



*Figure 4-1 Differences in operation steps for trace signals*

After the extraction, there are 190 features. Usually the features with such a large dimension contain vectors that are highly correlated and redundant. For more effective modeling, the feature dimension needs to be reduced. Common dimension reduction tools include PCA, ICA, Fisher Criterion, and Gain Ratio. Fisher Criterion and Gain Ratio are feature ranking methods. They are used to select features are will maximize the distance of data between healthy and faulty conditions. However, they require faulty data for training. In the situation for this research, only calibration data or healthy condition data can be used for modeling training. This makes PCA or ICA a better choice for dimension reduction. In this study, PCA is used for dimension reduction. The principles of PCA has been introduced in Section 2.2.1. The setting here for dimension reduction using PCA is: 3 PCs were selected according to the scree plot.

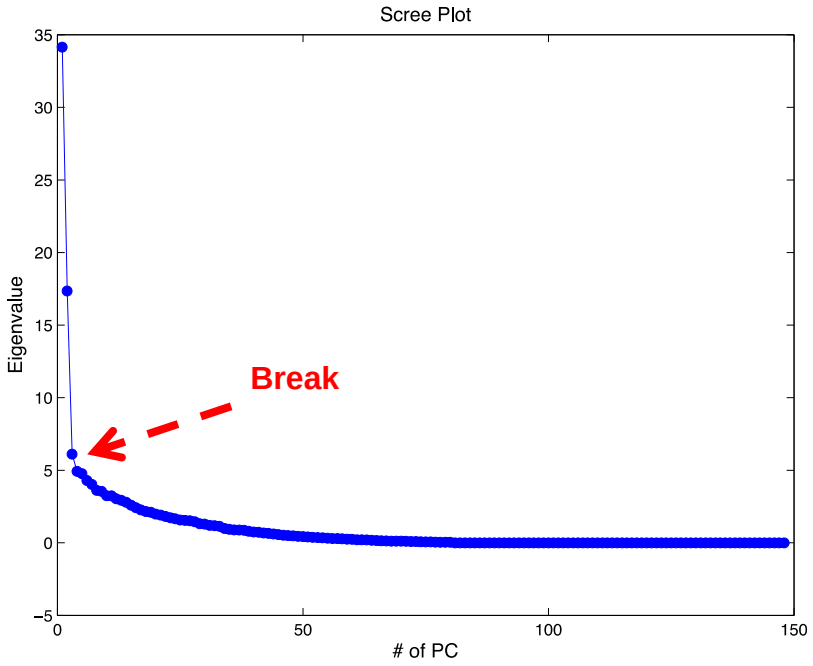



*Figure 4-2 Scree plot for LAM data PCA for number of PC selection*

## 4.3. Process Monitoring using Single Global Model

This section talks about conventional one-model-for-all approach as one of the comparing methods. The flowchart of single global model process monitoring approach is shown in Figure 4-3. During training stage, nominal condition data will go through pre-processing, feature extraction, and dimension reduction to a final feature matrix. Then the healthy condition features will be modeled using a global health model. In this study, the models adopted include: PCA-MSPC and SOM-MQE. The modeling process will generate a model for healthy state and a health limit. During monitoring stage, the monitored data will go through the same process with training data from pre-processing to reduced-dimension feature matrix. Then the trained model will be used to transfer the obtained feature matrix into a health metric. Comparison with the health metric will then decide which run is healthy, and which run is faulty.

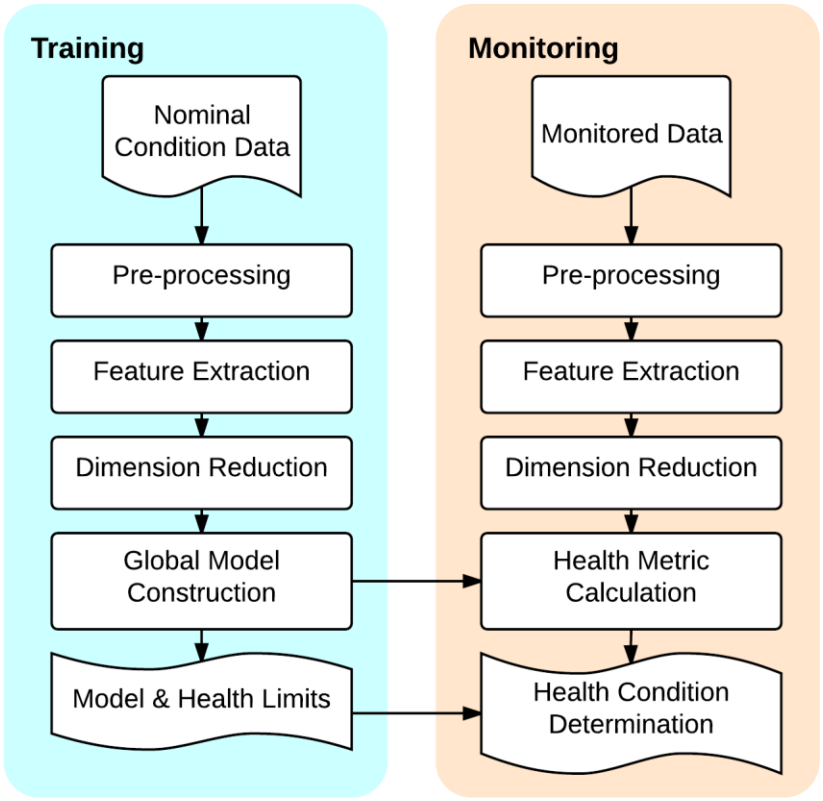



*Figure 4-3 Flowchart of single global model process monitoring approach*

The health monitoring result of PCA-MSPC is shown in Figure 4-4. Both Hotelling's $T^2$ and SPE are calculated as the health metric. As introduced in Section 4.2. , 3 principal components were chosen. To ensure that the best of PCA-MSPC method is investigated, the final fault detection rate was inspected against the number of PCs. It turned out that 3 PCs gave the highest fault detection rate for the metric of SPE. It can be seen from the figure that $T^2$ is performing very poorly, giving a fault detection rate of only 9.5238%, although there is no false alarm. On the contrary, SPE gives a decent fault detection rate of 76.1905%, and false positive rate of 15.3846%.

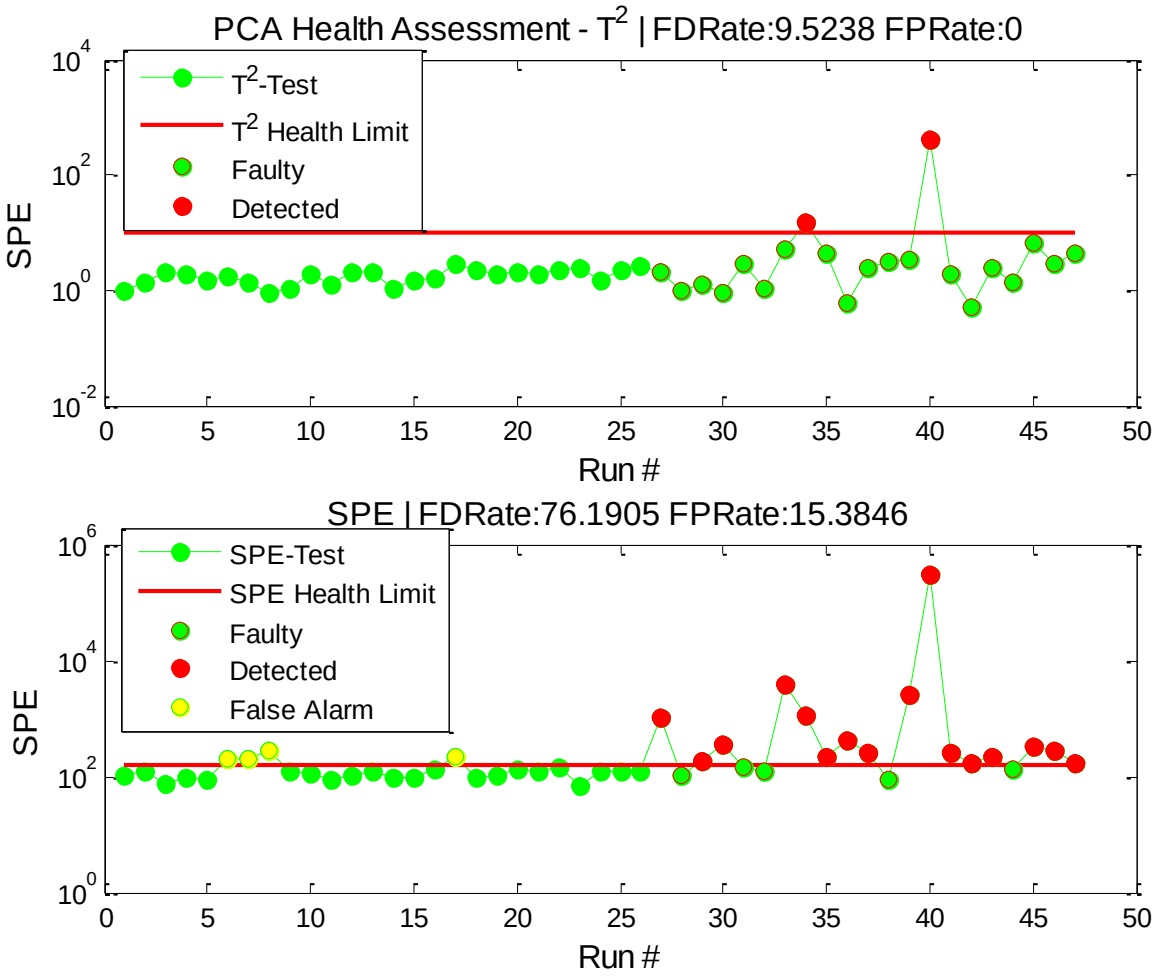



*Figure 4-4 Health monitoring for semiconductor etching process using PCA-MSPC*

Health monitoring result of SOM-MQE is shown in Figure 4-5. An SOM map was trained using only nominal condition data, which in this case is the complete feature matrix with bad values removed, and then the minimum quantization error (MQE) was calculated, and a health threshold defined based on 4 times the standard deviation of the training (healthy condition) MQE. After the SOM map was obtained and the health limit constructed, the newly monitored data were input into the already trained SOM map. Same with the training process, an MQE value was generated for every monitoring sample. The testing result is presented

in Figure 4-5. The fault detection rate and false positive rate are the same with global PCA-MSPC model, which is 76.1905% and 15.3846%, respectively.

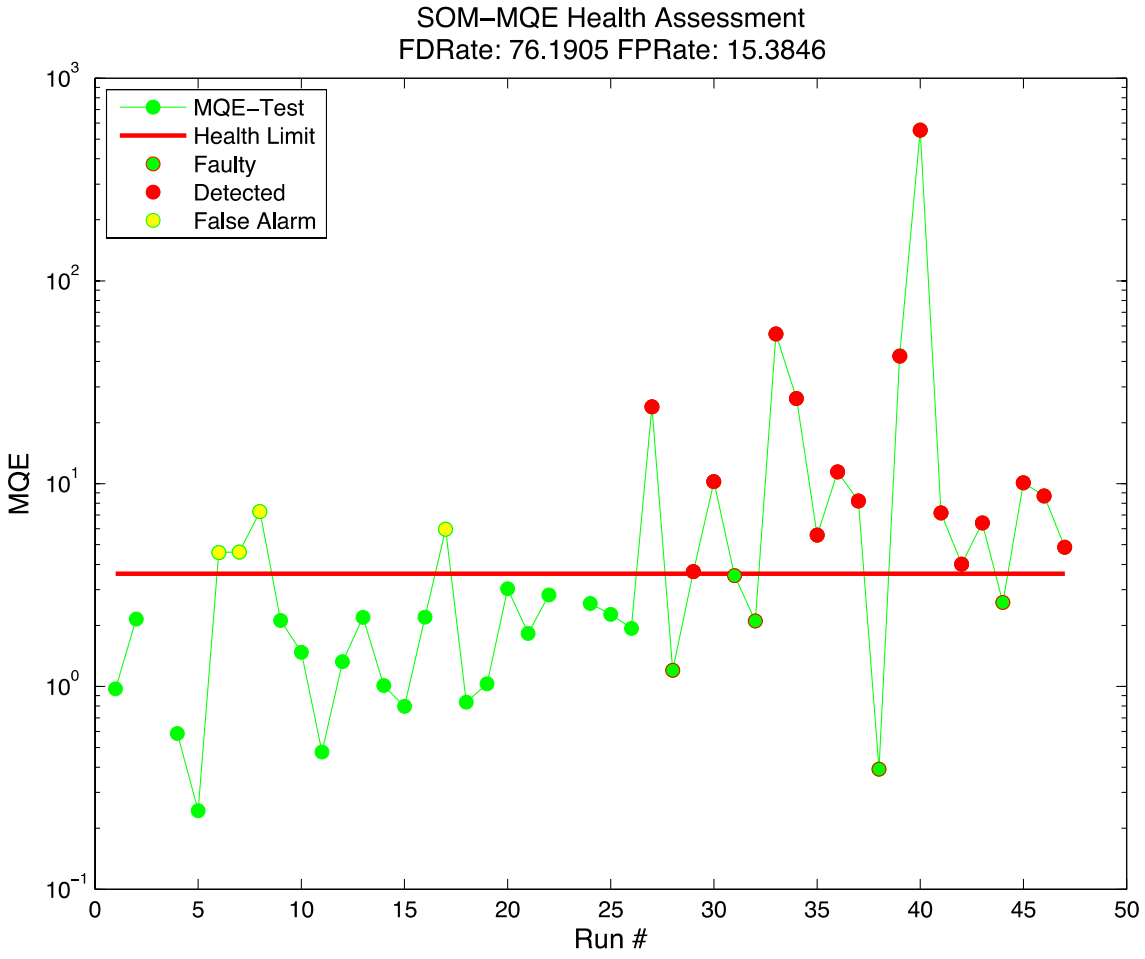



*Figure 4-5 Health monitoring for semiconductor etching process using SOM-MQE*

For both global models, the fault detection rate is no higher than 80%, and the false alarms are more than 15%. Besides, even though SOM is supposed to be a more advanced machine learning model, judging from the performance under the global modeling scheme, it does not outperform the simple PCA-MSPC approach. At this point, we can see that process drifts will have a significant negative influence on the model performance. Without taking into account

process drifts, the model will be highly likely to mistake normal condition for faults, and will not be able to recognize the pattern of normal condition well.

### 4.4. Process Monitoring using Multi-Regime Local Model

A flowchart of multi-regime local modeling for etching process monitoring is shown in Figure 4-7. The modeling approach consists of two aspects, which is the same with aforementioned global monitoring approaches.

The first aspects is training. During training stage, nominal condition data were again pre-processed and transformed into a feature matrix containing summary statistics of trace signals. What is different is that there is no dimension reduction process for local modeling, but all the features were taken as the input for a regime-recognizing model. In this research study, SOM was used as an effective clustering tool to learn and identify different drifting regimes. It should be noted that the nominal condition data contain all the three drifting regimes, and it is assumed that these three drifting scenarios will occur in a sequence-switching manner. For training, SOM separates the regimes by measuring the distances of the features from a reference plane. Each cluster is considered a regime. A codebook will then be generated to describe the core of each cluster. Then, within each cluster, a PCA-MSPC is constructed the same as the global modeling approach. Local nominal condition models were saved for each regime.

During monitoring stage, the monitored data were processed first, and features were extracted. Afterwards, a regime label is assigned to this monitored run by classifying the features to one of the recognized categories. After regime labeling, the features will be fed into the corresponding local PCA-MSPC model,

and a health index will be calculated for this particular run and compared with the local health limit. Eventually, a decision is made whether the monitored run is healthy or not.

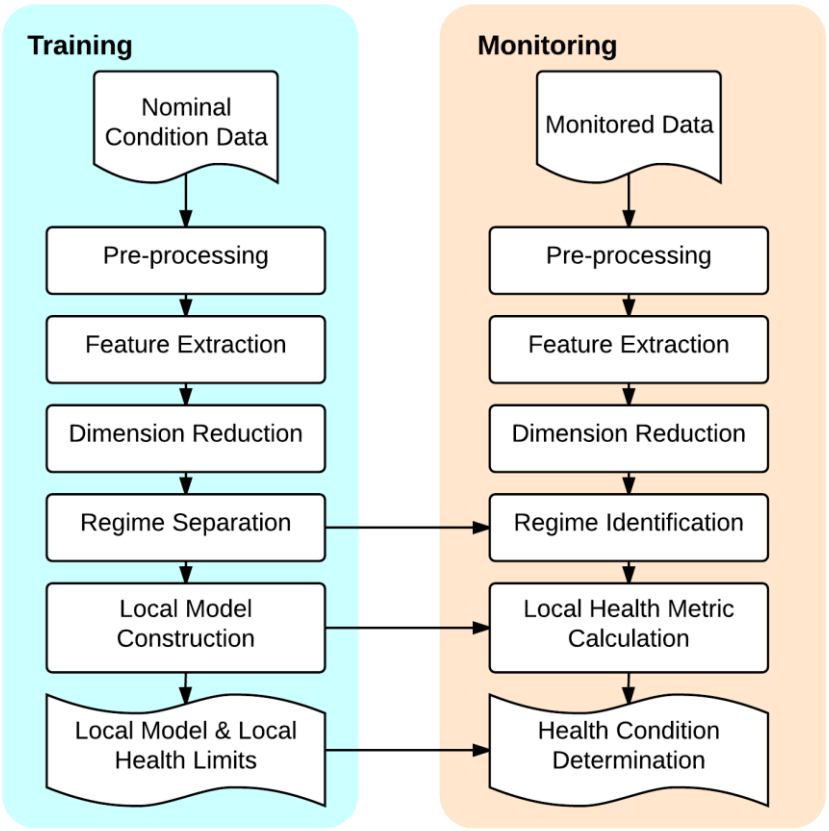



*Figure 4-6 Flowchart of multi-regime local model approach for semiconductor etching process monitoring*

The result of SOM+PCA is presented in Figure 4-7. The subplot on the top is the regime labeling result for testing data. We can see that during healthy condition, the regime is very well recognized, but for faulty condition data, the regimes are not well identified. This is because the regime-identifying model is trained using nominal condition data, and when the data faulty, the pattern of each regime will not be consistent with the nominal condition scenario.

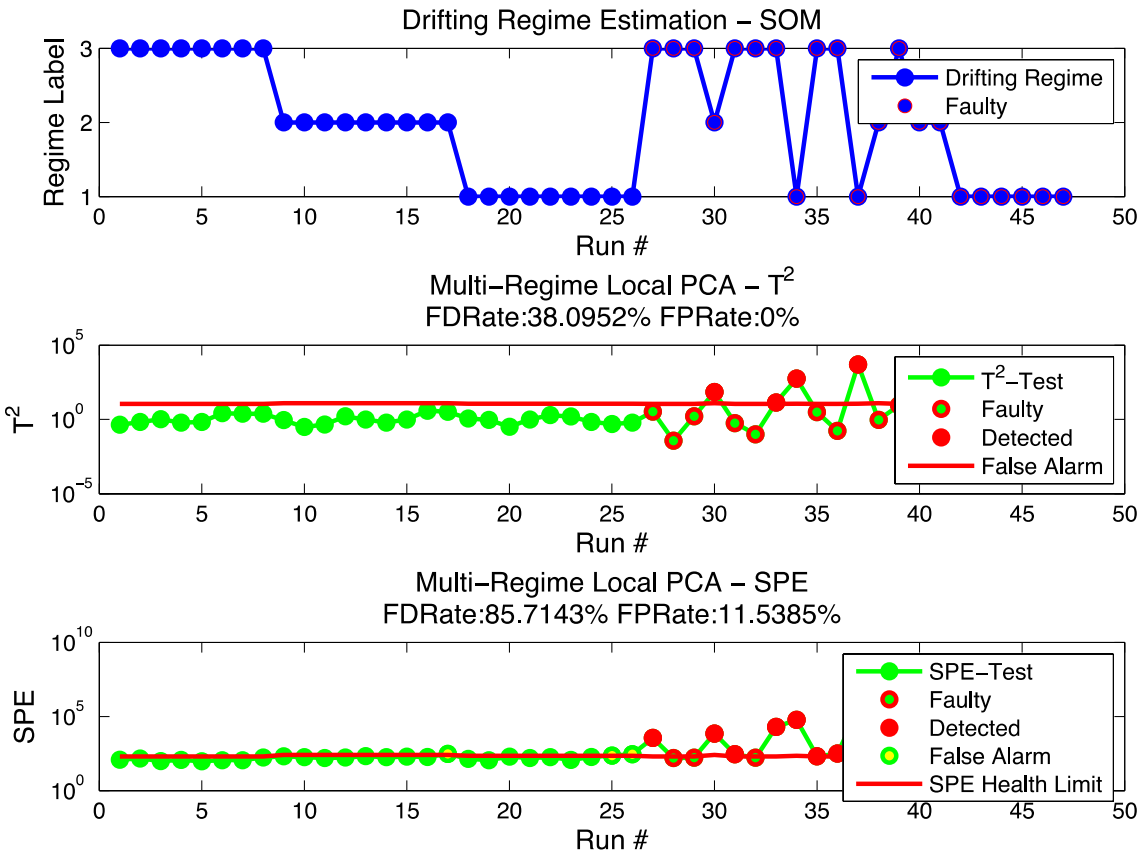



*Figure 4-7 Health monitoring for semiconductor etching process using multi-regime local model (SOM + PCA)*

However, the fault detection rate of both $T^2$ and SPE has increased significantly when PCA was applied locally for each regime - 38.0952% from 9.5238% for $T^2$, and 85.7143% from 76.1905% for SPE. The false positive rate of SPE also decreased from 15.3846% to 11.5385%. And the false positive rate for $T^2$ has remained at 0%. This result has proven the importance of modeling regime changes and switching between local models when assessing the health condition.

Another multi-regime local model tested was the local SOM-MQE model. Similar to the previous method, during training stage, trace data was pre-processed and features were extracted. Afterwards, the model used SOM as a

regime-identification tool to model the regime changes of the nominal condition data; then SOM was again adopted as a health assessment tool to capture the pattern of normal condition for each regime. At each regime, a different health limit was calculated based on 4 times the standard deviation of nominal condition MQE. During monitoring stage, the model will first identify the regime of the coming feature instances, and then apply the corresponding local model to calculate the MQE health index. Eventually, the monitored data MQE values will be compared with the local health limit to determine whether it is a faulty run or healthy run.

The result of multi-regime local SOM-MQE health monitoring is shown in Figure 4-8. The top subplot shows the regime labels of the testing data. We can see that this result is the same with SOM+PCA multi-regime modeling, since they are using the same modeling approach and algorithm for regime identification. The bottom plot shows the health index under multi-regime scheme. The health threshold can be seen to be switching from one to another as the identified regime changes. Eventually, this approach has achieved 95.2381% of fault detection accuracy, which is much higher than the original 76.1905%. However, the false positive rate was increased to 19.2308% from 15.3846%. This is possibly due to the low health limit. In order to eliminate the influence of the health limits and better benchmark different models, receiver operator curve (ROC) will be compared and discussed in the latter sections.

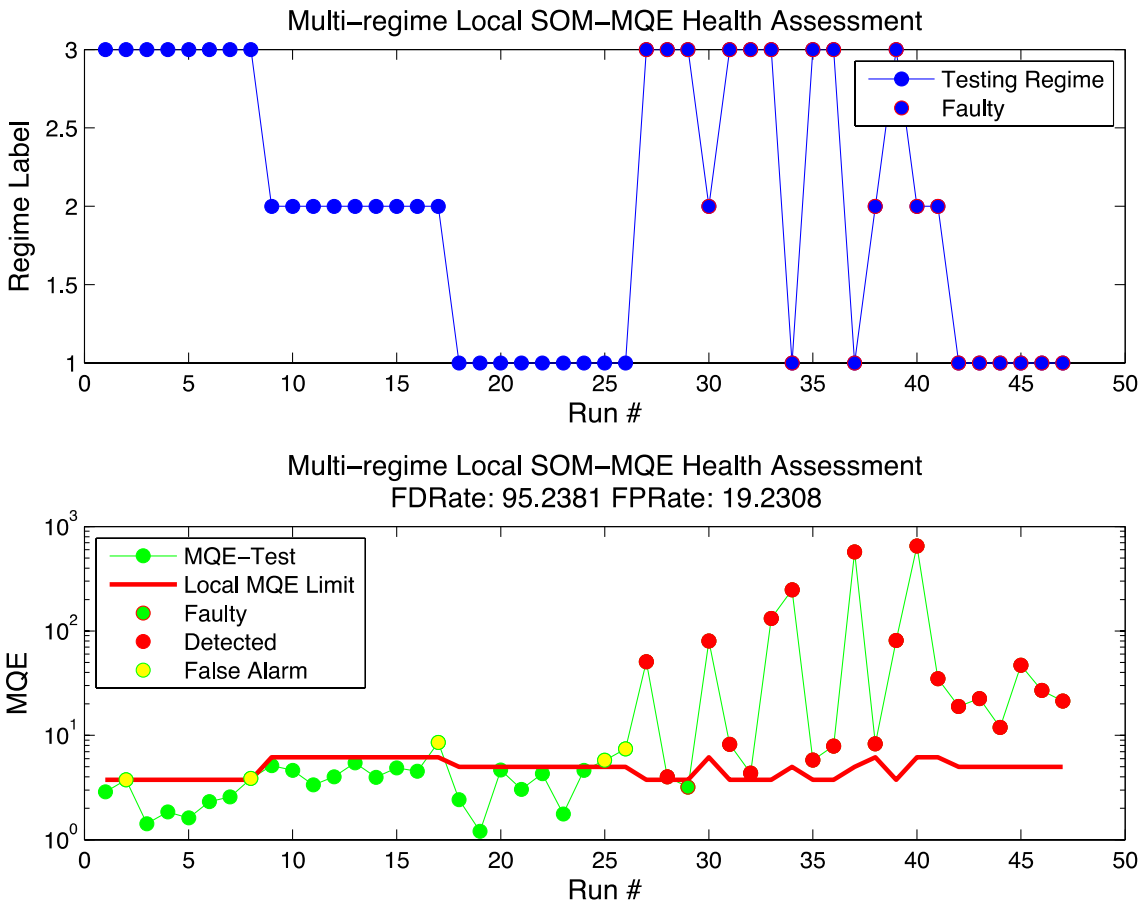



*Figure 4-8 Health monitoring for semiconductor etching process using multi-regime local model (SOM + SOM)*

In this section, a multi-regime modeling approach was adopted to monitor the etching process health condition. This fundamental difference from global modeling without taking into account regime changes has significantly increased the fault detection accuracy. However, it should be noted that when the features are faulty, the regime identification does not work well. And because the approach follows a “one-model-one-regime” structure, it means that a model from a regime that does not match with the monitored regime might be chosen to generate a health index. This risk also renders the result difficult to explain, and

makes it doubtful that even if a fault was correctly detected, it might be coincidence.

## 4.5. Application of Hidden Markov Model-Based Process Monitoring Approach

This section will focus on introducing the application of the proposed HMM-based process monitoring approach, and show results of the intermediate results to hopefully provide a thorough step-by-step guideline of how to use HMM for process monitoring.

The methodology has been explained in Chapter Chapter 3, and the detailed procedure depicted in Figure 3-1. This section will therefore not focus on the explanation of the procedure, but to emphasize some of the details that the author thinks need attention to correctly apply HMM. The following subsections will talk about dimension reduction, vector quantization, model initialization and how to cope with local maximum, which are all practical issues one will encounter when using discrete HMM.

### 4.5.1.Importance of Dimension Reduction

PCA has been introduced for a few times in this thesis. The function PCA and the number of PC chosen are explained in Section 4.2. . One function of dimension reduction particularly for discrete HMM modeling is that it reduces the chances of getting an empty clustering during vector quantization. This is important because it is assumed that different variables are statistically independent from each other when inputting into HMM, which means that the

probability of observing a certain symbol will be multiplied together over all the variables. If there are too many variables, the possibility that one of the symbols of a particular variable is not observed in the monitored sequence is high. And if it happens, zero observation probability will lead to negative infinity log-likelihood, which means that the training might be troublesome, or there will be no health index generated.

### 4.5.2.Vector Quantization Evaluation

Vector quantization (VQ) is a paramount step for discrete HMM modeling. The basic principles of VQ has been elaborated in Section 3.5. . One of the important topics of applying VQ is how to choose the number of symbols. Discrete HMM wants the minimum number of symbols to avoid empty clusters during observation (reasons explained in Section 4.5.1). In this light, in order to discretize the continuous time-series to a degree that loses minimum amount of information with minimum number of symbols, a metric will be needed to evaluate the “goodness-of-VQ”. A general measure of this is called the distortion rate. And a widely used metric is mean squared error (MSE). MSE will continuously decrease as the number of symbols becomes bigger. The number will be determined at the point when increasing the number of symbol will not make too much difference in MSE.

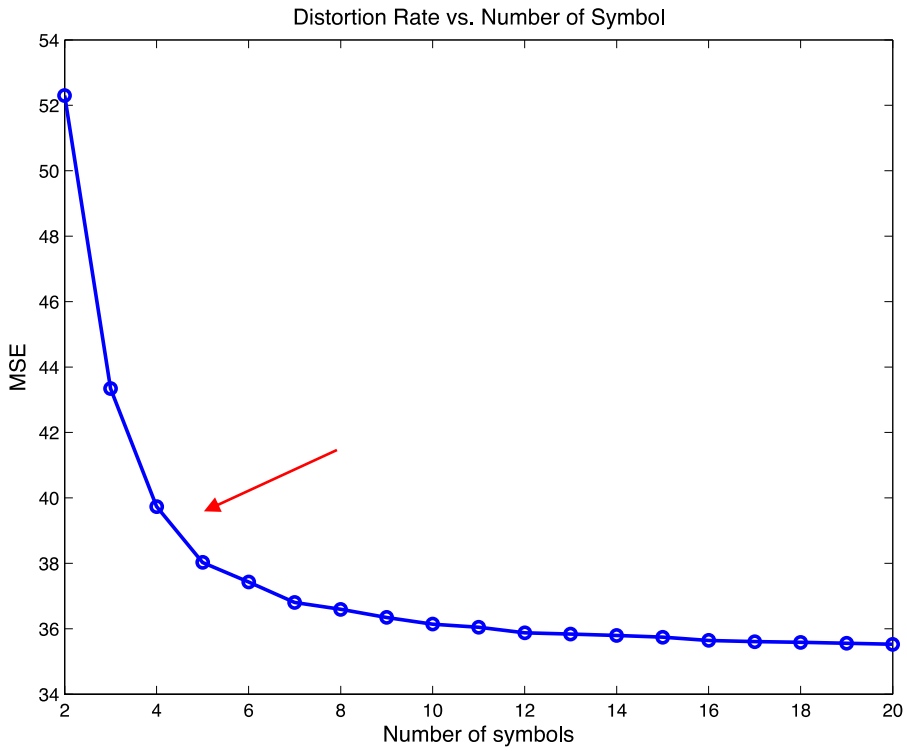



*Figure 4-9 Vector quantization distortion rate evaluation to choose the appropriate number of symbols*

The plot of MSE over the number of symbols chosen for VQ is shown in Figure 4-9. Judging from the plot, the best number of symbols will be 10 or 12 since after them the MSE does not significantly decrease anymore. However, the number of training instances is only 81, and the testing instances are even shorter. By following the rule of thumb (Mardia, Kent and Bibby, 1980) for choosing the number of clusters, an appropriate number would be $\sqrt{81/2} \approx 6$. Combining these evaluation results and apply them by trial and error to the model to avoid negative infinity log-likelihoods during training and testing, the number of symbols was determined to be 4.

From the above discussion, we can see that the process of choosing the number of symbols should not depend on merely one criterion. The distortion

rate, the avoidance of empty clusters, and the final HMM performance should be evaluated all together to choose an optimal size for the codebook. VQ itself is an independent topic, and sometimes it can be so cumbersome that it is usually not fully discussed in the literature when applying discrete HMM. This section is to systematically discuss the procedure of determining the size of codebook, and provide references and guidelines to the users who are interested in using discrete HMM.

### 4.5.3.Determining Model Structure

Before model initialization and training, the structure of HMM needs to be determined, including the type of HMM, and the number of states.

For model type, since the drifting states are assumed to be able to transit from one to another freely without specific directions, complete HMM is used in this study. In another word, one process drifting state can transit to any another states, including itself.

For the number of states, the aforementioned Bayesian information criterion (BIC) is used as the metric of "goodness of fit" for an HMM. The mathematical representation of BIC is:

$$BIC = -2\ln f(y|\widehat{q}) + k\ln n$$

where $lnf(y|\hat{\theta})$ is the log-likelihood, $k$ is the number of estimated parameters, and $n$ is the number of observations. Compared to Akaike information criterion (AIC) value, BIC is more stringent for models with larger number of parameters to estimate. From the equation itself, we can see that the smaller the BIC value, the

higher the Bayesian posterior probability. In this case study, we have obtained a BIC value graph shown in Figure 4-10. Apparently, 3 is the optimal number of states for the training data. And this matches with our idea of modeling the the three equipment drifting conditions as hidden states.

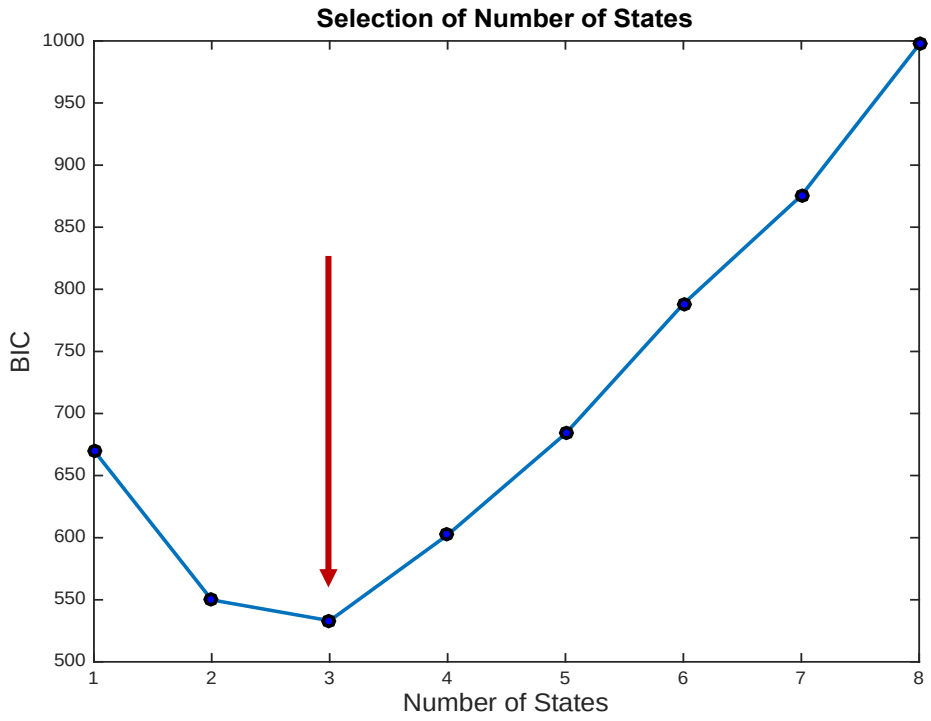



*Figure 4-10 Bayesian information criterion evaluation for number of states selection*

### 4.5.4.Model Initialization and Training

The training algorithm for HMM model, Baum-Welch algorithm, is locally optimal, meaning it can only converge to local maximum and might give a model that is not the best it can get, as illustrated in Figure 4-11. This means that the choice of “where to start the convergence”, namely, the initialization, will have an influential effect on the final result. On HMM parameter initialization and global optimum search, there are a large number of varying approaches, such as k-

means segmental HMM (Juang & Rabiner, 1990), Adaboost HMM feature enhancement (Yin, Essa, Starner and Rehg, 2008), and similarity matrix-based converging method (Clemente et al., 2012). Since at this phase of the study optimizing HMM initialization and model training is not the primary task, the method used in this paper to cope with local maximum convergence is to randomly initialize HMM parameters, iterate for 100 times, and then choose the model that possesses the highest log-likelihood. Even though it is computationally expensive, for offline training, obtaining an optimal model is the top priority and the computational efficiency is still within the acceptable range.

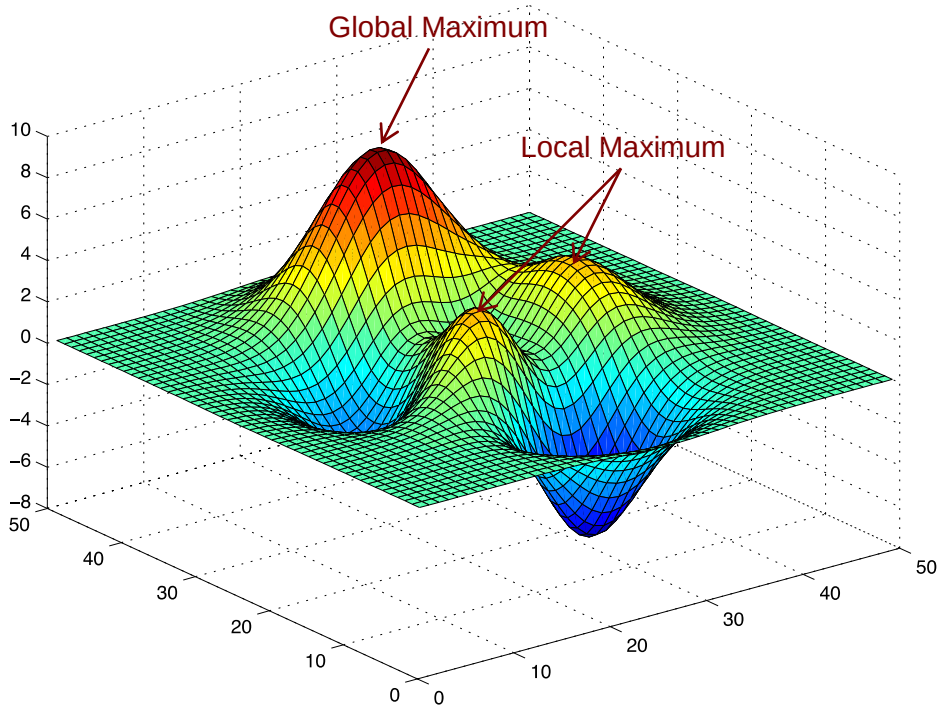



*Figure 4-11 Illustration of local optimum and global optimum*

### 4.5.5.Monitoring Result

The monitoring result of HMM-based process monitoring approach is shown in Figure 4-12. The top subplot indicates the testing result for regime identification. The state sequence is calculated using Viterbi algorithm.

It can be seen from the plot that during normal condition, the regime has been correctly identified; while during faulty condition, the regime is not well identified. This result is consistent with those from multi-regime modeling approaches. However, unlike the multi-regime local modeling approach, no local model has to be chosen for each regime. Instead, the probability of observing a certain observation (run) under the most possible hidden state (regime) is evaluated and represented by log-likelihood.

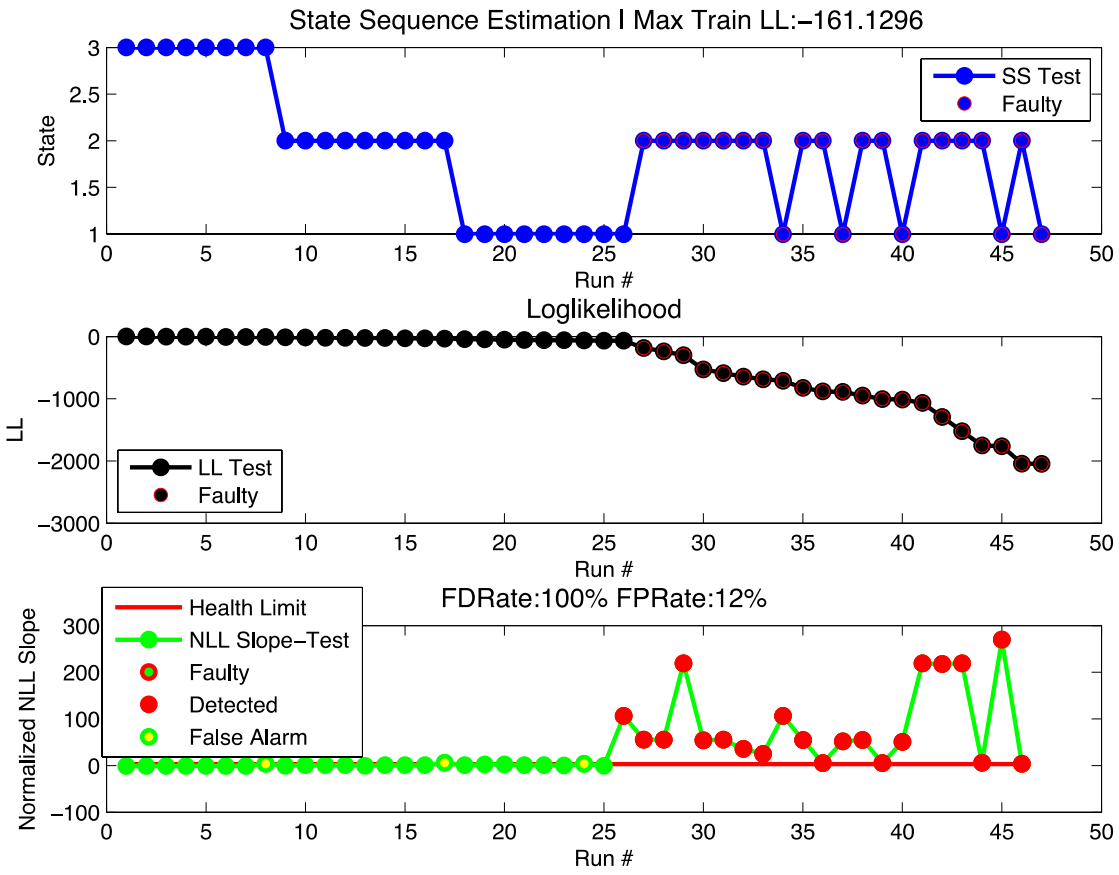



*Figure 4-12 Health monitoring for semiconductor etching process using HMM-based method*

From the middle subplot, we can see that during normal condition, log-likelihood steadily decreases as the sequence length grows longer. Yet when the faulty runs occur, log-likelihood values immediately starts to drop from the normal condition and begin to deviate. This is because the faulty observations do not conform to the initially trained model and the probability of observing these instances becomes lower than nominal condition. Eventually, log-likelihood was transformed to the normalized negative log-likelihood slope metric, which is introduced in Section 3.7. . The health threshold was set to be 3 times the nominal condition deviation. The final fault detection rate achieves 100%, with a false positive rate of 12%. We can see that most of the false alarms occur when

the model is switching hidden states estimation, which is consistent with the mechanism of the model: the model will change the state sequence estimation when the current observation probability drops. Overall, the model is performing very well for detecting all the faults with a decent false positive rate, which no other models were able to achieve.

## 4.6. Benchmarking and Discussion

In order to compare the process monitoring approaches without the influence of health threshold, ROC curve was plotted for each approach and the area under curve (AUC) was calculated as a metric for model goodness. The ROC curve comparison is shown in Figure 4-13. It can be seen from the graph that the ROC curve of the proposed HMM-based health monitoring approach is the closest to the upper left corner of the graph. Quantifiably, the AUC values are compared in Table 4-4. HMM-based health monitoring method has an AUC values of more than 0.99, while the highest of others is below 0.93. HMM-based health monitoring method has improved the conventional approach performances as much as approximately 16%, and it has reached 100% fault detection rate, whereas no other benchmarking algorithms has achieved.

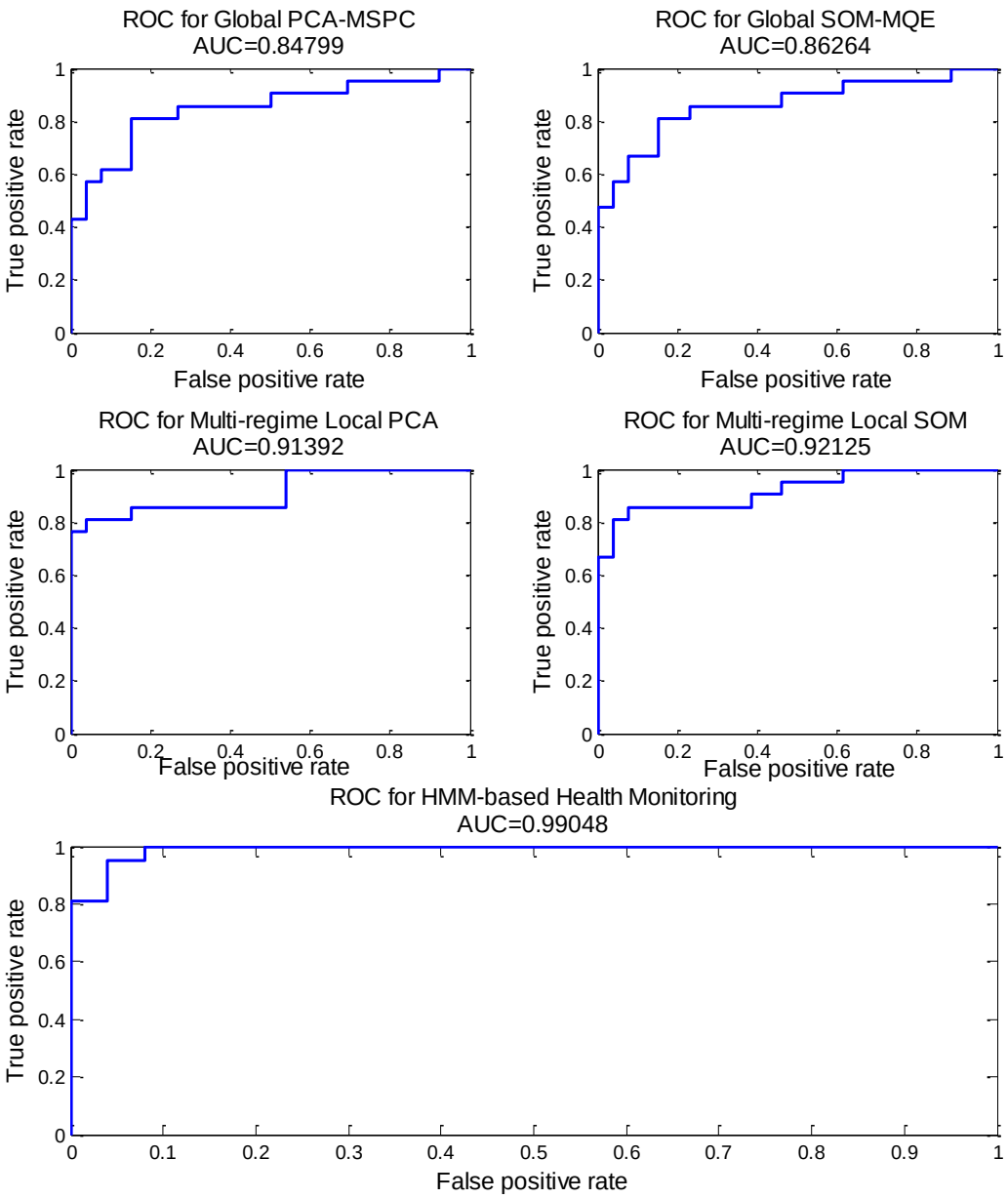



*Figure 4-13 ROC curve comparison between HMM-based health monitoring approach and other benchmarking approaches*

*Table 4-4 Comparison of AUC*

| Approach | AUC | Improvement [%] |
|---|---|---|
| **HMM** | 0.99048 | N/A |
| **Global PCA-MSPC** | 0.84799 | 16.8035 |
| **Global SOM-MQE** | 0.86264 | 14.8195 |
| **Multi-regime Local PCA** | 0.91392 | 8.3768 |
| **Multi-regime Local SOM** | 0.92125 | 7.5149 |

It should be noted that even if HMM-based process monitoring method has achieved the best performances, there are still false alarms generated when the state is transitioning from one to another. When the state is changing, before the model find the next most possible state, the likelihood of the observation will drop temporarily, but will return to normal after the condition stabilizes. This requires a health limit that is able to evaluate whether it is a regime change or an actual fault, which can be included in the future research tasks.

# Chapter 5. Conclusions and Future Work

## 5.1. Conclusions

Process monitoring is paramount for early detection of problems and guarantee product quality for a variety of applications such as chemical industry, semiconductor fabrication, and oil and gas industry. One of the key challenges is how to construct a robust health index against process drifts. Process drifts are formed for various reasons: it could be from upstream variation, form environmental condition changes, or from tool aging. In order to prevent process drifts from compromising the effectiveness of health indices, they need to be taken into account and be differentiated from faults. Conventional approaches deal with this issues using local models or switch between a library of models and select the one that best fits the monitored data.

This research study proposes a process monitoring approach based on HMM to tackle the problem of discrete drifting stages during monitoring. Normal condition data containing all the drifting scenarios were adopted for off-line modeling, when drifting stages are modeled as hidden states of HMM. During monitoring stage, the slope of the negative log-likelihood was normalized and taken as the health index. It is assumed that when the sequence length gets longer in normal condition, the log-likelihood slope will remain steady. Yet when faults occur, the log-likelihood will drop drastically, rendering a large slope deviation. A health threshold was set up also based on the normal condition

health index, and the determination of equipment health condition will be carried out by comparing the index with the threshold.

A case study for semiconductor etching process was adopted for methodology validation. Conventional methods of global PCA-MSPC, global SOM-MQE, multi-regime local PCA, and multi-regime local SOM are applied for benchmarking. Result from the proposed method was compared with those from the benchmarking approaches, and it was proved to have the best performance among all of the five models, and achieved 100% accuracy in fault detection rate.

## 5.2. Recommendations for Future Work

HMM is a mathematically rich model. The power of it lies in the fact that (1) it can be used to build a unified PHM framework for health assessment, diagnosis and prognosis; (2) it can be extended to various other forms such as hidden semi-Markov model, hierarchical hidden Markov model, mixture-of-Gaussian hidden Markov model, etc.; (3) convenience of incorporating physical knowledge and interpretation of model parameters.

However, besides its strengths, it also has its weaknesses. Since Baum-Welch converges constantly to local maximum, HMM is very sensitive to the initialized parameters. This issue is dealt with in this research merely by running the training process with randomized initial values for 100 times, and select the trained model with highest log-likelihood. Yet apparently, this method is not efficient and does not guarantee a global optimal result. Therefore, one of the future tasks is to develop or apply existing improved initialization method to HMM

so that a globally optimal parameter set can be obtained and computational load can be reduced.

Another aspect of improvement is the adaptivity of the approach and to develop an adaptive mechanism to automatically recognize process drift without having to include all drifting conditions in the training set. Current methods assumes that the drifting stages are discrete and are all available for modeling during training, which might not be true in practice. Therefore, an approach that is able to adapt to the conditional changes can be developed to better increase the robustness of the process monitoring approach.

Finally, prediction of faults can be developed as a further step for process monitoring method so that faults can be avoided. The proposed approach does not have the capability of prediction yet, but merely monitoring the health condition of the system. In practice, if early fault detection or prediction can be performed, users will then be able to avoid downtime and reduce cost of wasted materials by knowing before faults happening.

# Bibliography


Baruah, P. & Chinnam*, R. B. (2005). HMMs for diagnostics and prognostics in machining processes. *International Journal of Production Research*, vol. 43(6), pp. 1275-1293.

Bellman, R. (1956). Dynamic programming and Lagrange multipliers. *Proceedings of the National Academy of Sciences of the United States of America*, vol. 42(10), pp. 767.

Boser, B. E., Guyon, I. M. & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. *Proceedings of the fifth annual workshop on Computational learning theory* (144-152)

Bro, R. (1996). Multiway calidration. multilinear PLS. *Journal of chemometrics*, vol. 10, pp. 47-61.

Brown, A., Catterson, V., Fox, M., Long, D. & McArthur, S. (2007). Learning models of plant behavior for anomaly detection and condition monitoring. *Intelligent Systems Applications to Power Systems, 2007. ISAP 2007. International Conference on* (1-6)

Chen, Y. (2012). Data Quality Assessment Methodology for Improved Prognostics Modeling. PhD, University of CIncinnati.

Cholette, M. E. (2012). Performance monitoring and fault-tolerant control of complex systems with variable operating conditions.

Cholette, M. E., Celen, M., Djurdjanovic, D. & Rasberry, J. D. (2013). Condition Monitoring and Operational Decision Making in Semiconductor Manufacturing. *Semiconductor Manufacturing, IEEE Transactions on*, vol. 26(4), pp. 454-464.

Clemente, I. A., Heckmann, M. & Wrede, B. (2012). Incremental word learning: Efficient hmm initialization and large margin discriminative adaptation. *Speech Communication*, vol. 54(9), pp. 1029-1048.

Dayal, B. S. & MacGregor, J. F. (1997). Recursive exponentially weighted PLS and its applications to adaptive control and prediction. *Journal of Process Control*, vol. 7(3), pp. 169-179.

Dong, M., He, D., Banerjee, P. & Keller, J. (2006). Equipment health diagnosis and prognosis using hidden semi-Markov models. *The International Journal of Advanced Manufacturing Technology*, vol. 30(7-8), pp. 738-749.

George, B. (1994). Time Series Analysis: Forecasting & Control, 3/e, Pearson Education India.

Gray, R. M. (1984). Vector quantization. *ASSP Magazine, IEEE*, vol. 1(2), pp. 4-29.

Grubbs, F. E. (1969). Procedures for detecting outlying observations in samples. *Technometrics*, vol. 11(1), pp. 1-21.

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural networks*, vol. 4(2), pp. 251-257.

Infantesb, M. F. M. G. G. & Longa, D. (2004). Robot Introspection through Learned Hidden Markov Models.

Isermann, R. (1984). Process fault detection based on modeling and estimation methods—a survey. *Automatica*, vol. 20(4), pp. 387-404.

Jablonski, A. & Barszcz, T. (2013). Validation of vibration measurements for heavy duty machinery diagnostics. *Mechanical Systems and Signal Processing*, vol. 38(1), pp. 248-263.

Jabłoński, A., Barszcz, T. & Bielecka, M. (2011). Automatic validation of vibration signals in wind farm distributed monitoring systems. *Measurement*, vol. 44(10), pp. 1954-1967.

Jolliffe, I. (2002). Principal component analysis, Wiley Online Library.

Juang, B.-H. & Rabiner, L. (1990). The segmental K-means algorithm for estimating parameters of hidden Markov models. *Acoustics, Speech and Signal Processing, IEEE Transactions on*, vol. 38(9), pp. 1639-1641.

Kadlec, P., Gabrys, B. & Strandt, S. (2009). Data-driven soft sensors in the process industry. *Computers & Chemical Engineering*, vol. 33(4), pp. 795-814.

Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE*, vol. 78(9), pp. 1464-1480.

Ku, W., Storer, R. H. & Georgakis, C. (1995). Disturbance detection and isolation by dynamic principal component analysis. *Chemometrics and intelligent laboratory systems*, vol. 30(1), pp. 179-196.

Kumar, A., Tseng, F., Guo, Y. & Chinnam, R. B. (2008). Hidden-markov model based sequential clustering for autonomous diagnostics. *Neural Networks, 2008. IJCNN 2008.(IEEE World Congress on Computational Intelligence). IEEE International Joint Conference on* (3345-3351)

Kwan, C., Zhang, X., Xu, R. & Haynes, L. (2003). A novel approach to fault diagnostics and prognostics. *Robotics and Automation, 2003. Proceedings. ICRA'03. IEEE International Conference on* (604-609)

Larue, P., Jallon, P. & Rivet, B. (2011). Modified k-mean clustering method of HMM states for initialization of Baum-Welch training algorithm. *19th European Signal Processing Conference (EUSIPCO 2011)* (951-955)

Li, W., Yue, H. H., Valle-Cervantes, S. & Qin, S. J. (2000). Recursive PCA for adaptive process monitoring. *Journal of process control*, vol. 10(5), pp. 471-486.

Liu, Y. (2008). Predictive modeling for intelligent maintenance in complex semiconductor manufacturing processes, ProQuest.

Mardia, K. V., Kent, J. & Bibby, J. (1980). Multivariate analysis (probability and mathematical statistics), Academic Press London.

Murphy, K. P. (2012). Machine learning: a probabilistic perspective, MIT press.

Ning, C., Chen, M. & Zhou, D. (2014). Hidden Markov Model-Based Statistics Pattern Analysis for Multimode Process Monitoring: An Index-Switching Scheme. *Industrial & Engineering Chemistry Research*, vol. 53(27), pp. 11084-11095.

Qin, S. J. (1998). Recursive PLS algorithms for adaptive data modeling. *Computers & Chemical Engineering*, vol. 22(4), pp. 503-514.

Qin, S. J. (2012). Survey on data-driven industrial process monitoring and diagnosis. *Annual Reviews in Control*, vol. 36(2), pp. 220-234.

Qin, S. J. & McAvoy, T. J. (1992). Nonlinear PLS modeling using neural networks. *Computers & Chemical Engineering*, vol. 16(4), pp. 379-391.

Rabiner, L. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, vol. 77(2), pp. 257-286.

Rashid, M. M. & Yu, J. (2012). Hidden Markov model based adaptive independent component analysis approach for complex chemical process monitoring and fault detection. *Industrial & Engineering Chemistry Research*, vol. 51(15), pp. 5506-5514.

Sarkar, S., Jin, X. & Ray, A. (2011). Data-driven fault detection in aircraft engines with noisy sensor measurements. *Journal of Engineering for Gas Turbines and Power*, vol. 133(8), pp. 081602.

Schölkopf, B., Smola, A. & Müller, K.-R. (1997). Kernel principal component analysis. Artificial Neural Networks—ICANN'97, Springer**:** 583-588.

Smith, L. I. (2002). A tutorial on principal components analysis. *Cornell University, USA*, vol. 51, pp. 52.

Smyth, P. (1994). Hidden Markov models for fault detection in dynamic systems. *Pattern recognition*, vol. 27(1), pp. 149-164.

Somervuo, P. & Kohonen, T. (1999). Self-organizing maps and learning vector quantization for feature sequences. *Neural Processing Letters*, vol. 10(2), pp. 151-159.

Suykens, J. A. & Vandewalle, J. (1999). Least squares support vector machine classifiers. *Neural processing letters*, vol. 9(3), pp. 293-300.

Tipping, M. E. (2001). Sparse Bayesian learning and the relevance vector machine. *The journal of machine learning research*, vol. 1, pp. 211-244.

Tobon-Mejia, D., Medjaher, K., Zerhouni, N. & Tripot, G. (2011). Hidden Markov models for failure diagnostic and prognostic. *Prognostics and System Health Management Conference (PHM-Shenzhen), 2011* (1-8)

Tobon-Mejia, D. A., Medjaher, K., Zerhouni, N. & Tripot, G. (2012). A data-driven failure prognostics method based on mixture of gaussians hidden markov models. *Reliability, IEEE Transactions on*, vol. 61(2), pp. 491-503.

Venkatasubramanian, V., Rengaswamy, R., Yin, K. & Kavuri, S. N. (2003). A review of process fault detection and diagnosis: Part I: Quantitative model-based methods. *Computers & chemical engineering*, vol. 27(3), pp. 293-311.

Viterbi, A. J. (1967). Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *Information Theory, IEEE Transactions on*, vol. 13(2), pp. 260-269.

Wang, X., Kruger, U. & Irwin, G. W. (2005). Process monitoring approach using fast moving window PCA. *Industrial & Engineering Chemistry Research*, vol. 44(15), pp. 5691-5702.

Wise, B. M., Gallagher, N. B., Butler, S. W., White, D. D. & Barna, G. G. (1999). A comparison of principal component analysis, multiway principal component analysis, trilinear decomposition and parallel factor analysis for fault detection in a semiconductor etch process. *Journal of Chemometrics*, vol. 13(3-4), pp. 379-396.

Wold, H. (1985). Partial least squares. *Encyclopedia of statistical sciences*.

Yang, J., Xu, Y. & Chen, C. S. (1994). Hidden Markov model approach to skill learning and its application to telerobotics. *Robotics and Automation, IEEE Transactions on*, vol. 10(5), pp. 621-631.

Yang, T. & Xu, Y. (1994). Hidden Markov Model for Gesture Recognition, CMU-RI-TR-94 10, Robotics Institute, Carnegie Mellon Univ., Pittsburgh, PA.

Yin, P., Essa, I., Starner, T. & Rehg, J. M. (2008). Discriminative feature selection for hidden markov models using segmental boosting. *Acoustics, Speech and Signal Processing, 2008. ICASSP 2008. IEEE International Conference on* (2001-2004)

Yu, J. (2010). Hidden Markov models combining local and global information for nonlinear and multimodal process monitoring. *Journal of process control*, vol. 20(3), pp. 344-359.